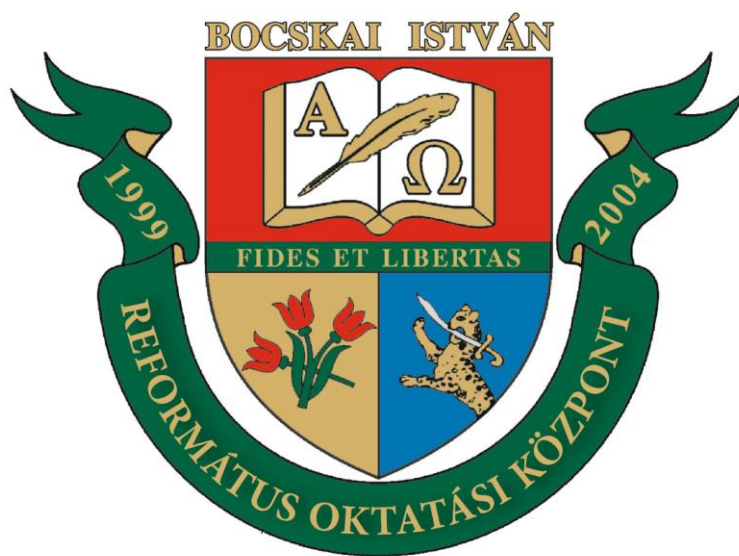


# ***THE SETUP CONFIGURATOR***



## ***Csapattagok:***

- ***Egri Roland Imre***
- ***Hazafi Levente***
- ***Kovács Norbert Kristóf***

**Szoftverfejlesztő, Szoftvertesztelő  
(2025-2026)**

# *Gyorshivatkozások/Tartalomjegyzék*

|                                                   |    |
|---------------------------------------------------|----|
| 1. Ábra: Az oldalsó panel megnyitása              | 7  |
| 2. Ábra: A négy darab részletes szűrési szempont. | 9  |
| 3. Ábra: Összes termék                            | 9  |
| 4. Ábra: termék kártyák.                          | 10 |
| 5. Ábra: Spotlight                                | 11 |
| 6. Ábra: Klikk animáció                           | 12 |
| 7. Ábra: Gyors navigáló menü részlet              | 12 |
| 8. Ábra: Ranglista                                | 13 |
| 9. Ábra: Products Tábla                           | 16 |
| 10. Ábra: Portok és Kábelek                       | 19 |
| 11. Ábra: Kontroller Kód                          | 24 |
| 12. Ábra: Endpointok                              | 25 |
| 13. Ábra: Progress érték                          | 25 |
| 14. Ábra: Build kód                               | 26 |
| 15. Ábra: Szinkronizálás                          | 26 |
| 16. Ábra: Kapcsolatok                             | 27 |
| 17. Ábra: Meta adatok és dinamikus szűrés         | 27 |
| 18. Ábra: Biztonság és szűrés                     | 28 |
| 19. Ábra: NgFor kód                               | 32 |
| 20. Ábra: Lapozási rendszer                       | 33 |
| 21. Ábra: Keresési és szűrési rendszer            | 34 |
| 22. Ábra: Kategória alapú szűrés                  | 35 |
| 23. Ábra: Oldalsó termék panel                    | 36 |
| 24. Ábra: Felhasználói Rangrendszer               | 37 |
| 25. Ábra: Spotlight és FeaturedPool kód           | 38 |
| 26. Ábra: Dock kód                                | 38 |
| 27. Ábra: Animációk és vizuális effektek          | 39 |
| 28. Ábra: Háttér kód                              | 40 |
| 29. Ábra: Konfigurációs központ, Kártyák          | 41 |
| 30. Ábra: Eszköz tulajdonságlap                   | 43 |
| 31. Ábra: Setup menü                              | 44 |
| 32. Ábra: Setup jobbkattintás                     | 44 |
| 33. Ábra: Tulajdonság sáv kinézete                | 45 |
| 34. Ábra: Az első kép a felépítésről              | 46 |

|                                         |    |
|-----------------------------------------|----|
| 35. Ábra: A második kép a felépítésről  | 46 |
| 36. Ábra: A harmadik kép a felépítésről | 47 |
| 37. Ábra: A negyedik kép a felépítésről | 48 |
| 38. Ábra: Az ötödik kép a felépítésről  | 49 |
| 39. Ábra: Profilos Mysetup              | 49 |
| 40. Ábra: Modell szűrő                  | 52 |
| 41. Ábra: Hibás adatú kereső            | 54 |
| 42. Ábra: Oldal léptető                 | 54 |
| 43. Ábra: Jegyzetömb                    | 56 |
| 44. Ábra: AI                            | 58 |
| 45. Ábra: Értesítés                     | 58 |
| 46. Ábra: Messages                      | 59 |
| 47. Ábra: Beállítások                   | 59 |
| 48. Ábra: További Beállítások           | 60 |
| 49. Ábra: Admin oldal                   | 61 |
| 50. Ábra: Admin - User panel            | 61 |
| 51. Ábra: Admin - Product panel         | 62 |
| 52. Ábra: Admin - Reports panel         | 63 |
| 53. Ábra: Admin - Log panel             | 63 |
| 54. Ábra: Admin - System panel          | 64 |
| 55. Ábra: Admin - Setup panel           | 64 |

## 1. Bevezetés

A projektötletünk elsősorban úgy jött létre, hogy összeülve megbeszéltük egymás érdeklődési köreit, illetve ötleteit hogy ki milyen témájú, típusú oldalt képzelt el. Úgy képzeltük el, hogy egy olyan projektek/oldal készítőnk, ami összefügg, az érdeklődési körünkkel amennyire csak lehet, hogy aztán ne legyen véleményeltérés, vagy bármi panasz a másik féltől.

Először készítettünk egy alapot az oldalhoz, és mindenki felvetette a saját ötleteit, hogy ki milyen funkciót szeretne belerakni, és ha az ötlet jónak hangzott, vagy ha volt összefüggés és ötlet mögötte, az belekerült az oldalra. A kezdeti tervezési folyamat során több különböző megoldást is megvizsgáltunk, és igyekeztünk olyan irányba vinni a projektet, amely technikailag megvalósítható, ugyanakkor érdekes és hasznos is lehet a felhasználók számára. Emellett figyelembe vettük azt is, hogy milyen funkciók tudják legjobban bemutatni a projektünkben alkalmazott technológiákat és fejlesztési megoldásokat.

Mindenképpen ügyeltünk arra, hogy felosszuk arányosan a munkát egymás között, és úgy írjuk meg a kódokat, hogy más kódrészeibe ne zavarjon bele. A feladatok elosztása során mindenki kapott egy-egy nagyobb részt, amelyért felelősséget vállalt, így a fejlesztés folyamata átláthatóbb és szervezettebb lett. Ennek köszönhetően a különböző funkciók fejlesztése párhuzamosan is tudott haladni, anélkül hogy a kódban ütközések vagy problémák jelentkeztek volna.

A következő látható mintákon, és kódokon úgy vannak felépítve, hogy egymás kód részleteibe ne zavarjon bele, vagy ne okozzon bármilyen zavarást. A fejlesztés során igyekeztünk modulárisan felépíteni az oldalt, vagyis az egyes funkciók és komponensek külön kezelhetők legyenek, így a későbbi módosítások vagy bővítések is egyszerűbben elvégezhetők. Ez a megközelítés segített abban is, hogy a hibakeresés és a tesztelés átláthatóbb legyen.

Eredetileg ez egy tervezői oldalnak készült, majd átalakítottuk arra, hogy mindenki a saját termék, setup, hangszer, autóról feltöltheti az adatait az oldalra a megtervezett setup-okon kívül. Az átalakítás során az volt a célunk, hogy az oldal ne csak egy egyszerű tervezőfelület legyen, hanem egy olyan platform, ahol a felhasználók megoszthatják egymással a saját összeállításait, ötleteiket és konfigurációikat is. Így az oldal nemcsak egyéni használatra alkalmas, hanem közösségi jelleggel is rendelkezik, ahol más felhasználók setupjai és termékei is megtekinthetők.

A projekt megvalósítása során külön figyelmet fordítottunk arra, hogy az oldal felépítése logikus és könnyen átlátható legyen a felhasználók számára. Ennek érdekében a funkciókat és az oldal különböző részeit úgy terveztük meg, hogy egyszerűen használhatóak legyenek, és a felhasználók gyorsan megtalálják azokat az információkat vagy lehetőségeket, amelyeket keresnek. A célunk az volt, hogy egy olyan weboldalt hozzunk létre, amely nemcsak technikai szempontból működik megfelelően, hanem felhasználói élmény szempontjából is könnyen kezelhető és áttekinthető.

## 1.1 A projekt célja

Ebben a részben a projekt alapvető céljai kerülnek bemutatásra.

A projekt létrehozásának célja, hogy segítséget nyújtson a felhasználók számára az igényeiknek megfelelő eszközök megtalálásában saját setupjaik kialakításához. Ide tartozhat például autó, hangszer vagy zenei szoftver, komplett házimozi-rendszer, számítógép vagy annak alkatrészei, illetve a hozzájuk tartozó perifériák.

Ezek a célok azért kerültek meghatározásra, mert a projekt készítői a valós életben is érdeklődnek a setupok építése és tervezése iránt, valamint szívesen foglalkoznak azzal, hogy alapos utánajárás alapján segítsék egymást a megfelelő eszközök kiválasztásában.

A weboldal célja, hogy ezeket a folyamatokat mindenki számára egyszerűbbé és átláthatóbbá tegye. Emellett fontos szempont volt, hogy a kiválasztási és tervezési folyamat ne csak gördülékeny, hanem vizuálisan is élvezhető legyen. Így a setup megtervezése nem válik unalmassá vagy kényszerű feladattá, hanem egy olyan folyamattá, amely valóban élvezhető, és lehetőséget ad az elképzelt setup megvalósítására.

Az optimalizált szűrők célja, hogy megkönnyítsék azoknak a termékeknek és kategóriáknak a keresését, amelyek valóban relevánsak az adott felhasználó számára. A főoldal és a szűrési lehetőségek segítségével a kínálat személyre szabható, így mindenki könnyebben megtalálhatja a számára megfelelő elemeket.

A **MySetup** fülön kizárólag a saját setupok jelennek meg, illetve itt van lehetőség új setup létrehozására is. Az elkészült setupok igény szerint módosíthatók, törölhetők, bővíthetők vagy finomíthatók.

A **Terv** oldalon a tervezett setupok és eszközök tekinthetők meg, valamint ezek is szabadon módosíthatók az egyéni elképzeléseknek megfelelően.

Az **Üzenetek** fül lehetőséget biztosít arra, hogy a felhasználók kapcsolatba lépjenek egymással, segítséget kérjenek, illetve ötleteket osszanak meg egymással.

Az **AI funkció** gyors és kényelmes segítséget nyújt a setupok megtervezésében, valamint tanácsokat ad a már meglévő vagy tervezett összeállításokkal kapcsolatban.

Ezek a funkciók együttesen hozzájárulnak a projekt céljának megvalósításához.

A projekt további célja, hogy ne csupán egyedi eszközök kiválasztására legyen alkalmas, hanem különféle rendszerek megtervezésére is lehetőséget biztosítson, például hangrendszerek vagy UTP-hálózatok esetében. Ennek köszönhetően nemcsak eszközök, hanem teljes rendszerek is megtervezhetők a felületen.

## 1.2 A probléma bemutatása

A projekt létrehozásának egyik fő indoka az volt, hogy a setupok és eszközök igényeknek megfelelő kiválasztásával járó bonyodalmak csökkenthetők legyenek, és a folyamat sokkal felhasználóbaráttá váljon. Ezt segíti elő az a lehetőség is, hogy a saját setupok elmenthetők.

Ennek köszönhetően könnyebben nyomon követhető, hogy egy adott számítógéphez milyen alkatrészek kerültek kiválasztásra, milyen hangrendszerek lettek megtervezve, illetve milyen autók vagy hangszerek szerepelnek az adott setupban. Így elkerülhető, hogy valaki elfelejtse vagy pontatlanul idézze fel, milyen eszközöket választott korábban.

A setupok összeállításának egyik legnagyobb nehézsége, hogy mindenkinek más igényei vannak, ugyanakkor sokan nem tudják önállóan megtalálni a saját szempontjaiknak leginkább megfelelő, megbízható eszközöket.

Egy átgondolt és megfelelő döntés meghozatalához általában alaposabban bele kell mélyedni az adott eszközök tulajdonságaiba és lehetőségeibe. A projekt ezt a folyamatot kívánja lerövidíteni részletes szűrési és keresési funkciók segítségével.

Az adatbázisba bekerülő eszközök egyenként, gondos utánajárás alapján kerültek kiválasztásra. Ez biztosítja, hogy a felhasználók megbízható és releváns lehetőségek közül válogathassanak.

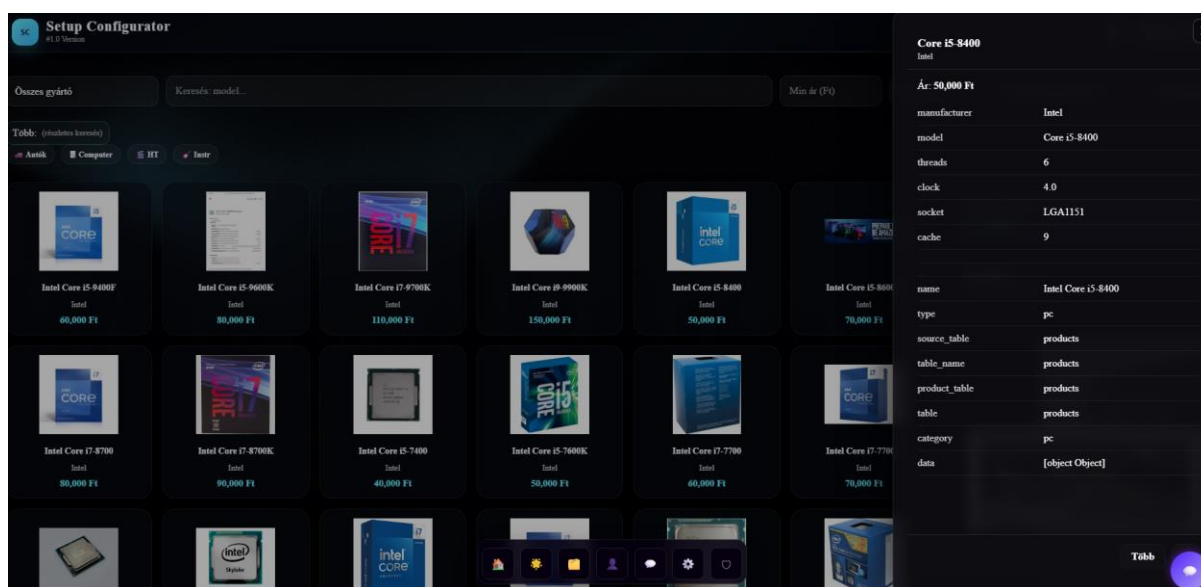
A tervezői nézetben lehetőség van a preferált elemek kiemelésére is, így jól láthatóvá válik, hogy mely eszközök élveznek nagyobb prioritást az adott setupban.

Az is fontos előny, hogy több setup is létrehozható, ami lehetőséget ad a különböző alkatrészekkel és összeállításokkal való kísérletezésre. Ezáltal könnyebben megtalálhatók az egyéni igényekhez legjobban illeszkedő eszközök és setupok.

## 2.1 Funkcionális követelmények

Norbi:

Projektrészem egyik fő funkciója a felhasználó regisztráció után lehet látni termék kártyákat, amikre rányomva megjelenik egy oldal(side) panel, amin az adott termék részletei/adatai találhatóak. A Home oldalon látható egy animációs háttér, ami az oldal kinézetéhez viszonyuljon.



### 1. Ábra: Az oldalsó panel megnyitása

Külön a termékeket lehet rendezni ár alapján növekvő, illetve csökkenő sorrendbe, lehet A-Z-ig növekvő, vagy csökkenő sorrendbe, van az alapértelmezett rendezés, külön egy option fülbe lehet gyártó alapján szűrni, külön Min-Max input mezőből is lehetséges a szűrés, és végül a felhasználó a Kereső input-ba maga írhatja be, hogy melyik gyártóra, modellre szeretne rákeresni.

De ha a felhasználó csak egy bizonyos téma alapján szeretne szűrni, akkor erre készítettem négy darab fő szűrési szempontot, amik a következők:

- Autók
- Házimozik
- Hangszerek
- Számítógép

Ezen szűrők gombra rányomva csak azokat a termékeket írja ki, ami abba a szűrőbe tartozik. Ezenfelül az adott gombokon belül vannak még részletesebb szűrői tulajdonságok, amivel még jobban le lehet keresni/kérdezni az adott terméket.

Több: (részletes keresés)

Autók

Computer

HT

Instr

Autók

Csak autó táblákra szűr

Gyártó (pl. BMW)

Modell (pl. 320d)

Ár min (Ft)

Ár max (Ft)

Kivitel (összes)

Lóerő min

Lóerő max

0–100 min (s)

0–100 max (s)

Ülések min

Ülések max

Üzemanyag (összes)

Évjárat min

Évjárat max

Váltó (összes)

Autó szűrők törlése

A Keresés gomb ezeket is figyelembe veszi.

PC

Meta-alapú (dinamikus) PC szűrő

PC típus (összes)

Gyártó (összes)

Modell

Ár min

Ár max

PC szűrők törlése

A Keresés gomb ezeket is figyelembe veszi.

HT

Meta-alapú (dinamikus) HT szűrő

HT típus (összes)

Gyártó (összes)

Modell (pl. Q990D)

Ár min

Ár max

HT szűrők törlése

A Keresés gomb ezeket is figyelembe veszi.

Instr

Egyszerű szűrő (meta: gyártók + táblák)

Minden

Tábla (összes)

Gyártó (összes)

Modell

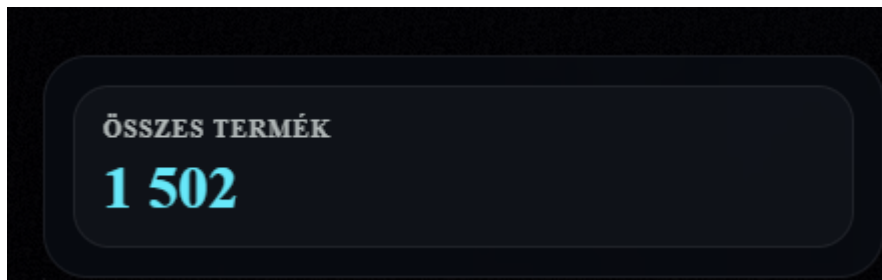
Instr szűrők törlése

A Keresés gomb ezeket is figyelembe veszi.

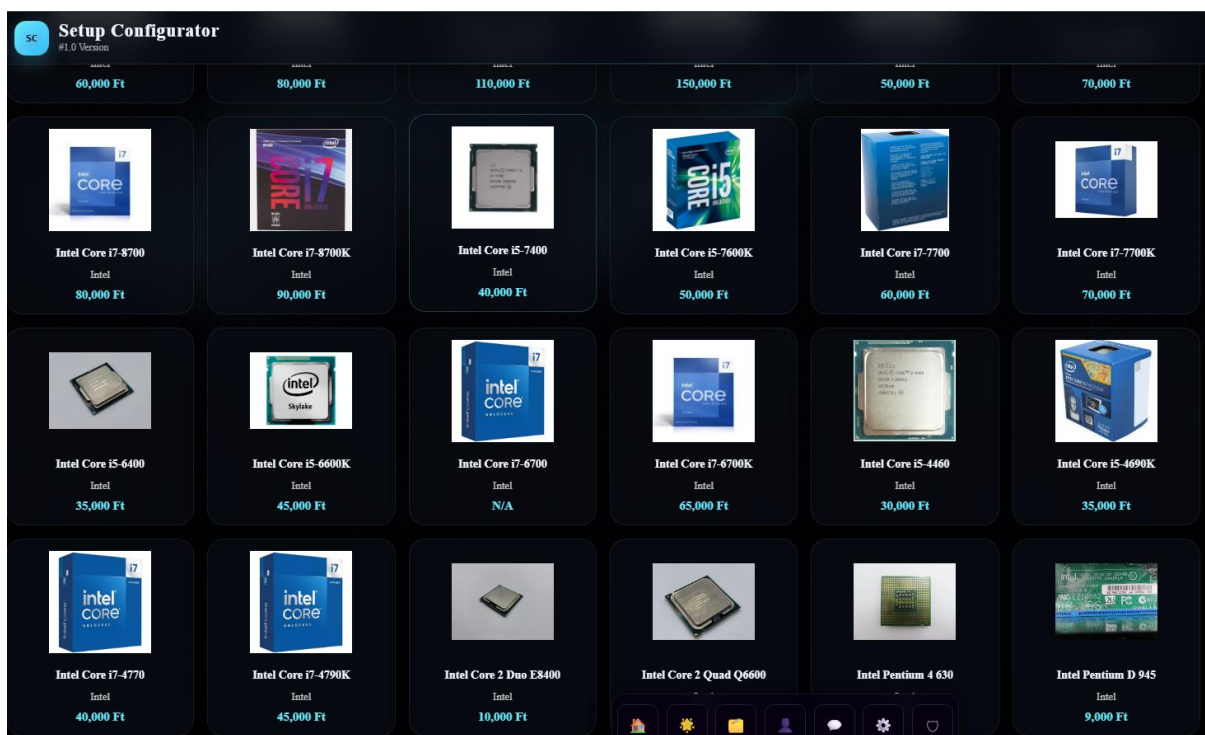
## 2. Ábra: A négy darab részletes szűrési szempont.

Az oldalunk bal oldala felé található a termékek(products-ok), összesen 1502 darab termék található, persze ez nem egyszerre van megjelenítve, hanem szekciókra van

bontva, és egy ilyen szekcióban 50 product van megjelenítve, hogy ne akadjon(laggoljon) az oldal. Az oldal alján lévő nyíl ikonnal jelzett navigációs gombokkal lehet a következő, illetve az előző 50 termékre menni.



**3. Ábra: Összes termék**



**4. Ábra: termék kártyák.**

Oldalunk jobb oldalán található egy "Spotlight" termékajánló. Ez a termékajánló az összes termék közül randomizálja be a termékeket, egy termék egy bizonyos ideig látszik, majd átvált egy következő random termékre. Ezek az átváltások egy animációval van megfűszerezve a hatékonyabb kinézetért. Szintén működik itt is az, hogy ha rányom az adott

spotlight termékre, kiírja az összes adat részleteit. Illetve a Spotlight fölött egy counter van, ami egy beépített animációval megszámlálja az adatbázisba lévő összes terméket.

**Spotlight**  
Random termék ajánló

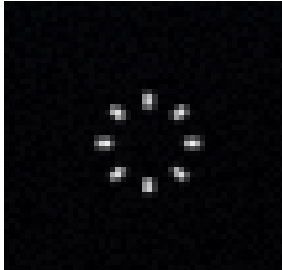


**JAM PEDALS**  
**JAM Pedals Pink Flow**  
**401 000 Ft**  
table: products id: 1212

|              |              |
|--------------|--------------|
| manufacturer | JAM Pedals   |
| model        | Pink Flow    |
| effect_type  | multi-effect |
| instrument   | guitar       |
| format       | pedal        |

*5. Ábra: Spotlight*

Az oldal látszólagos feldobása érdekében tettem be egy olyan animációt, ami az egér klikkelését jelzi. Egy kis szemantikai látvány miatt.

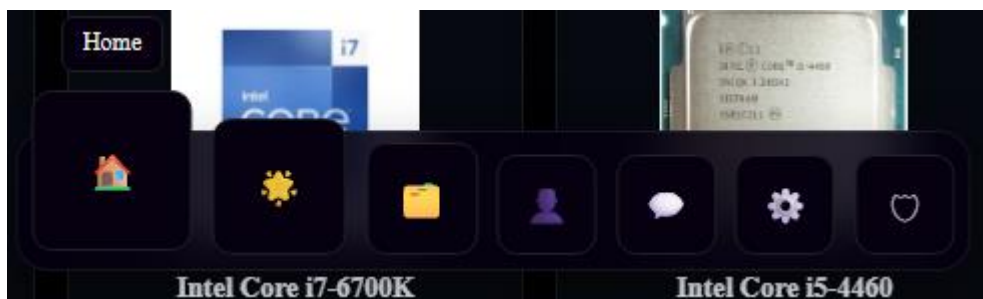


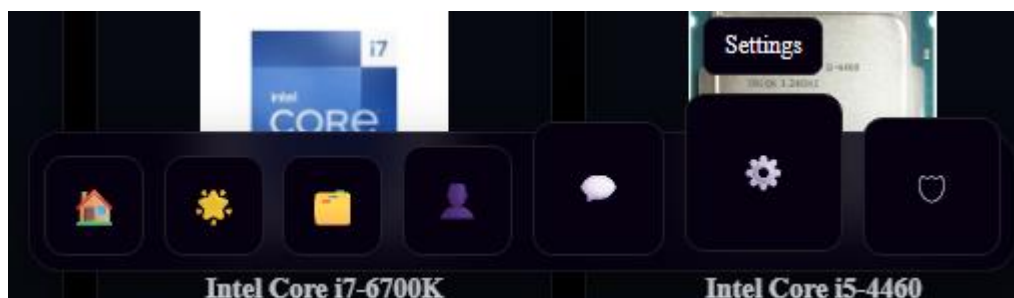
## 6. Ábra: Klikk animáció

Az oldal alján szintén egy animáció által beimportált “gyorshivatkozó menürészlet” található, ezek a hivatkozók a következő részekből állnak:

- Home
- Favourites
- MySetup
- Profile
- Messages
- Settings
- Admin

Ha valamelyik gombra rányom a felhasználó, akkor arra az oldalra fog vezetni.





**7. Ábra: Gyors navigáló menü részlet**

A profil oldalon belül egy rangrendszer is kialakításra került. A felhasználók különböző pontokat szerezhetnek például termékek feltöltésével, módosításával vagy törlésével stb... A megszerzett pontok alapján a felhasználók magasabb rangot érhetnek el, amelyek különböző vizuális effektekkel jelennek meg.

Illetve a Favorites, és a Setups menüben lévő "Konfigurációs Központ"-ban is egy külön animációs háttér található, hogy különleges szemantikai kinézettel szolgáljon.

Csináltam mellé egy "Leaderboard" oldalt, amire a profil oldalról lehet hozzáférni, ha rányom a felhasználó a "Ranglista" gombra, akkor megnyílik az új oldal, ahol látni lehet az összes felhasználót, illetve a pontjait, és ez egy eredménylista, hogy kinek mennyi pontja van, hanyas szinten áll, illetve az első három külön helyezésben is részesülnek.

| Helyezés | Felhasználó                         | Rang     | Pont  | Következő rang | Haladás                      |
|----------|-------------------------------------|----------|-------|----------------|------------------------------|
| #1       | Hazafi Levente<br>@HazafiLe         | Level 10 | 20000 | Max            | Max pont<br>10000 / 20000    |
| #2       | Egri Roland Imre<br>@EgriRol        | Level 1  | 100   | Level 2        | 151 pont kell még<br>0 / 211 |
| #3       | Királi Dénes<br>@KiráliDenes        | Level 1  | 100   | Level 2        | 151 pont kell még<br>0 / 211 |
| #4       | Kovács Norbert (compe)<br>@NorbiKov | Level 1  | 100   | Level 2        | 151 pont kell még<br>0 / 211 |
| #5       | Hazafi Levente<br>@HazafiLe         | Level 1  | 100   | Level 2        | 151 pont kell még<br>0 / 211 |
| #6       | Tótyi<br>@Totty                     | Level 1  | 100   | Level 2        | 151 pont kell még<br>0 / 211 |

**8. Ábra: Ranglista**

## 3.2 Architektúra leírása

A rendszer alapvetően két fő részből áll: egy Angular frontendből (Setup-Configurator mappa) és egy Node.js + Express backendből (backend mappa). A két oldal egymástól függetlenül fut, és REST API-n keresztül kommunikálnak egymással, JSON formátumban. A backend végzi az adatkezelést, a hitelesítést, a jogosultságok ellenőrzését és az AI hívásokat is, a frontend pedig csak a megjelenítésért és a felhasználói műveletekért felel. Az egészet a gyökérben egy közös package.json fogja össze, így egy paranccsal egyszerre indítható a backend és az Angular dev server.

A backend rétegezett felépítést követ, hogy átlátható és könnyen bővíthető maradjon. A server.js indítja el a HTTP szerveret, az app.js pedig az Express alkalmazást konfigurálja (cookie parser, CORS, JSON body, statikus képkiszolgálás). A forráskód funkciók szerint külön mappákba van szervezve: a routes mappában vannak a végpontok témakörök szerint (termékek, auth, profile, setup, leaderboard, messages, admin, AI stb.), a controllers mappában a HTTP kérésekre válaszoló függvények, a services mappában a tényleges üzleti logika (Supabase kapcsolat, termékkezelés, jogosultságok, pontszámítás, mailer, logger), a middlewares mappában pedig a jogosultságkezelő közbülső függvények (verifyUser, verifyAdmin, verifyAdminPlus, verifyOwner).

A frontend komponens alapú, és az src/app/Components mappán belül logikai csoportokra van bontva, hogy ne legyen egy gigantikus, áttekinthetetlen mappa. A főbb részek a header (minden oldal tetején), a Home (főoldal a Spotlight ajánlóval és a terméklistával), az Auth (bejelentkezés, regisztráció, authguard), a Settings, a Usersites (a bejelentkezett felhasználói oldalak: Setup, Plan, Profil, Messages, Admin), a Services (backenddel kommunikáló szolgáltatások), a Shared (újrahasználható közös komponensek) és a Product (termékgaléria és részletek).

Ezekén kívül van még külön mappa a Models, a Constants és a HTTP interceptor számára, ami minden kérésre automatikusan ráteszi a hitelesítési sütit.

Az adatbázis nem a projekt mellett fut, hanem egy külső Supabase példányon, amely PostgreSQL alapú. A backend a services/supabase.js modulon keresztül kapcsolódik hozzá service role kulccsal, így a kliens sosem fér hozzá közvetlenül az adatbázishoz. Az AI integráció szintén külső szolgáltatáson keresztül megy (Gemini), a képeket pedig a backend a saját datas mappájából szolgálja ki egy statikus /images végponton.

A hitelesítés JWT alapon működik, a token egy HttpOnly sütibe kerül, így a kliens oldali JavaScript nem fér hozzá. A frontend interceptora minden kérésre automatikusan ráteszi a

sütit, a backend pedig a védett route-okon middleware-ekkel ellenőrzi, hogy a felhasználó be van-e jelentkezve, és van-e megfelelő rangja a művelethez.

Egy tipikus kérés tehát úgy zajlik, hogy a felhasználó valamelyik komponensből elindít egy műveletet, az interceptor és a proxy átirányítja a kérést a backendre, ott lefutnak a védelmi middleware-ek, majd a controller meghívja a megfelelő service-t, ami a Supabase-en keresztül elvégzi a műveletet. Az eredmény visszafelé halad ugyanezen a láncon, és a frontend frissíti a UI-t.

### 3.4 Technológiai környezet

A tervezést egy közös Google Drive-on és egy Discord szerveren bonyolítottuk. A Drive-ba kerültek az ötletek, a vázlatok és a vizuális effektek tervei, a Discordon pedig folyamatosan tudtunk egyeztetni, ha valamit menet közben kellett megbeszélni. A projekt vége felé többször ültünk le közösen, mert a maradék funkciók nagyobb részét csak közös átbeszélés után tudtuk értelmesen lefejleszteni – ezek voltak a legidőigényesebb és legbonyolultabb részek.

A forráskódot GitHub-on tároltuk, a fejlesztést pedig WebStorm környezetben végeztük. Eredetileg egy egyszerű HTML, CSS és JavaScript alapú weboldalt készítettünk teszt jelleggel, hogy a logikát és az alap funkciókat ki tudjuk próbálni. Később áttértünk Angular keretrendszerre, ami TypeScripttel dolgozik. A meglévő JavaScript részeket egy kis utánajárással át tudtuk írni TypeScriptre, így nem kellett mindent nulláról újrakezdeni.

A frontendet az Angular keretrendszerrel készítettük, mivel modern és komponens alapú webalkalmazásokra lett kitalálva. Ennek köszönhetően a felületet külön komponensekre tudtuk bontani, amelyek egymástól függetlenül kezelhetők és újrafelhasználhatók – például a header gyakorlatilag minden oldalon megjelenik.

A nyelv TypeScript, ami a JavaScript kibővített változata, típusellenőrzéssel, így a hibák nagy részét már fejlesztés közben kiszűri. A megjelenítéshez HTML-t és CSS-t használtunk.

A backend Node.js környezetben fut, Express keretrendszerrel. Feladata az adatok kezelése, az adatbázissal való kommunikáció és az API végpontok biztosítása a frontend felé. Az architektúra REST API alapú, vagyis a kliens és a szerver HTTP kéréseken keresztül cserél adatot, JSON formátumban.

Az adatbázis nem a projekt mellett fut, hanem egy külső Supabase példányon, amely PostgreSQL alapú. A PostgreSQL azért volt jó választás, mert megbízható, széles körben elterjedt és nagy adatmennyiséget is stabilan kezel. Bár a fejlesztés alatt akadtak kisebb fennakadások, a rendszer jelenleg 100+ táblával is gond nélkül működik.

A képeket egy saját scraping megoldással, Bing képkereső segítségével töltöttük le, mivel a 980+ termékhez kézzel képeket keresni rendkívül időigényes lett volna. Ahol a Bing rossz vagy nem megfelelő képet adott vissza, ott manuálisan kicseréltük. A képek a backend gyökere mellett, egy datas mappában vannak tárolva, és egy külön végponton (/images) keresztül szolgáljuk ki őket.

A projektbe egy AI integráció is bekerült, a Gemini 2.5 Flash modellt használjuk (ingyenes változat). Bár ez egy kisebb nyelvi modell, a feladatainkhoz teljesen elegendő: a felhasználó rákérdezhet a termékek áraira, az AI hozzáfér az adatbázis adataihoz, ismeri az eszközök alapvető működését, valamint a felhasználó saját setupjait is látja, így személyre szabott javaslatot tud adni.

A megjelenés feldobására 3D elemeket is integráltunk, ezeket erre szakosodott oldalakról, ingyenes asset-ekként töltöttük le. Eredetileg az volt a tervünk, hogy minden eszköznek saját 3D modellje legyen, de ezt a jelenlegi keretek között nem tudtuk megvalósítani: nem minden termékhez érhető el ingyen használható modell, a hiányzókat pedig nekünk kellene megtervezni, ami önmagában rengeteg időt venne el. Ezért a 3D elemek egyelőre csak hangsúlyos pontokon jelennek meg, de a későbbiekben szeretnénk teljesen lefedni vele a termékeket.

### 3.1 Adatbázis-tervezés

Az adatbázis egy olyan rendszert szolgál ki, ahol a felhasználó különböző setup-okat tud építeni (pl. házimozi, PC stb.), és az adatbázis ezt teljesen ki is szolgálja, és akár a setup-okon belül össze is tudod a különböző eszközöket kapcsolni, mint egy házimozi esetében ahogy erősítőt és hangszórókat köt össze a felhasználó egy rca kábellel

Az egész adatbázis rendszer kb. úgy működik, mint egy LEGO:

- vannak termékek (products)
- ezeknek vannak tulajdonságaik (properties)
- ezekhez konkrét értékek (values) tartoznak
- és ezeket rakod bele egy setup-ba

ennek a megoldásnak van egy neve is EAV modell (Entity–Attribute–Value) és ez a modellben a products, properties és a values tábla.

| id | name                | category | type |
|----|---------------------|----------|------|
| 1  | Intel Core i5-9400F | pc       | cpu  |
| 2  | Intel Core i5-9600K | pc       | cpu  |
| 3  | Intel Core i7-9700K | pc       | cpu  |
| 4  | Intel Core i9-9900K | pc       | cpu  |
| 5  | Intel Core i5-9400  | pc       | cpu  |
| 6  | Intel Core i5-8600K | pc       | cpu  |
| 7  | Intel Core i7-8700  | pc       | cpu  |
| 8  | Intel Core i7-8700K | pc       | cpu  |
| 9  | Intel Core i5-7600  | pc       | cpu  |
| 10 | Intel Core i5-7600K | pc       | cpu  |
| 11 | Intel Core i7-7700  | pc       | cpu  |
| 12 | Intel Core i7-7700K | pc       | cpu  |
| 13 | Intel Core i5-6400  | pc       | cpu  |
| 14 | Intel Core i5-6600K | pc       | cpu  |
| 15 | Intel Core i7-6700  | pc       | cpu  |
| 16 | Intel Core i7-6700K | pc       | cpu  |
| 17 | Intel Core i5-4460  | pc       | cpu  |
| 18 | Intel Core i5-4590K | pc       | cpu  |
| 19 | Intel Core i7-4770  | pc       | cpu  |

## 9. Ábra: Products Tábla

a Képen a products tábla látszik, eleinte még minden külön termékre volt külön tábla, és bennük a tulajdonságaik. A jelenlegi táblában pedig a products ok elsőnek fel vannak sorolva, és nem külön táblákban, mint régebben. Ez azért lett így megoldva, mert így sokkal egyszerűbb kezelni és bővíteni a rendszert. A régebbi megoldás egy idő után nagyon átláthatatlanná vált, és nehéz volt új dolgokat hozzáadni. Emiatt lett átalakítva a rendszer egy általánosabb megoldásra.

A jelenlegi rendszerben a products tábla csak az alap adatokat tartalmazza:

- az eszköz neve
- kategóriája
- típusa
- és kategóriája

A részletes tulajdonságok nem itt vannak, hanem külön táblákban. A properties tábla tartalmazza azokat a tulajdonságokat, amik egy eszközhöz tartozhatnak (pl. gyártó, modell, teljesítmény stb.).

A values tábla pedig összeköti a product-ot a property-vel, és megadja a konkrét értéket. Ez azt jelenti, hogy például:

- product: hangfal
- property: teljesítmény
- value: 120W

Ez egy külön sor lesz a values táblában.

Ez a megoldás azért jó, mert:

- nem kell mindenhez külön oszlop
- bármikor lehet új tulajdonságot hozzáadni
- sokkal rugalmasabb az egész rendszer

A setup-ok kezelésére külön rendszer van.

A setups tábla tárolja magát a setup-ot (pl. "Nappali házimozsi"), és azt is, hogy melyik felhasználóhoz tartozik.

Egy felhasználónak több setup-ja is lehet, ez egy one-to-many kapcsolat (1 felhasználó → több setup). A setup\_devices tábla azt tárolja, hogy egy adott setup-ban milyen eszközök vannak elhelyezve.

Itt már nem csak az van benne, hogy milyen eszköz, hanem az is, hogy:

- hol van (pos\_x, pos\_y)
- milyen irányba áll (rotation)

Ez azért kell, mert a rendszer vizuálisan is megjeleníti a setup-ot.

A setup\_connections tábla kezeli az eszközök közötti kapcsolatokat.

Ez mondja meg például, hogy:

- melyik eszközből
- melyik másik eszközbe
- milyen porton keresztül
- milyen kábellel

van összekötve.

Ez már egy kicsit olyan, mint egy gráf:

- eszköz
- kapcsolat

The top screenshot displays the 'cables\_info[Cables]' table in the Supabase Table Editor. The table has 18 rows and 7 columns: id, cable\_category, cable\_type, standard, cable\_structure, connector\_a, and connector\_b. The data includes various cable types like Ethernet, Cat1, Cat2, Cat3, Cat4, Cat5, Cat5e, Cat6, Cat6a, Cat7, Cat8, HDMI 1.0, HDMI 1.1, HDMI 1.2, and connectors like RJ11, RJ45, and GG45.

The bottom screenshot displays the 'port\_types' table in the Supabase Table Editor. The table has 19 rows and 2 columns: id and name. The data includes various port types like HDMI, DisplayPort, Mini DisplayPort, Micro DisplayPort, USB-A, USB-B, USB-C, Micro USB, Mini USB, USB 3.0, USB 3.1, USB 3.2, USB4, Thunderbolt 1, Thunderbolt 2, Thunderbolt 3, Thunderbolt 4, VGA, and DVI-D.

## 10. Ábra: Portok és Kábelek

A portok és kábelek külön vannak kezelve ahogy képeken is látszik:

- setup\_device\_ports → az adott eszköz portjai
- port\_types → milyen port típusok léteznek (HDMI, USB stb.)
- cables\_info → a kábelek adatai

Ez azért lett külön szedve, hogy pontosan lehessen modellezni a valós kapcsolatokat (pl. nem lehet HDMI-t RCA-ba dugni).

## 4. Adatbázis-tervezés

Az adatbázis fő feladata, hogy kiszolgálja a setupok építését: termékeket tároljon, ezekhez tulajdonságokat és értékeket kapcsoljon, valamint kezelje a felhasználók által létrehozott setupokat és az eszközök közötti kapcsolatokat. Ezt egy külső Supabase példányon valósítottuk meg, ami PostgreSQL alapon működik.

A tervezés első verziójában minden termékkategóriának (pl. hangfal, router, CPU, autó, hangszer) saját külön táblája volt, és az adott eszköz minden tulajdonsága ennek a táblának egy-egy oszlopában szerepelt. Eleinte ez logikusnak tűnt, mert egyértelmű volt, hogy melyik adat hova tartozik. Ahogy viszont nőtt a termékek és kategóriák száma, ez a megoldás egyre kezelhetetlenebbé vált. Ha új tulajdonságot akartunk hozzáadni, azt minden érintett táblába külön be kellett volna írni; ha új kategória jött be, akkor egy teljesen új táblát kellett létrehozni a hozzá tartozó oszlopokkal együtt. Egy idő után 100+ táblánál tartottunk, amiben az átláthatóság teljesen elveszett, és minden új funkció jelentősen több munkát igényelt. Emiatt döntöttünk úgy, hogy átalakítjuk a rendszert egy általánosabb, rugalmasabb modellre.

A jelenlegi rendszer alapja az EAV modell (Entity–Attribute–Value), ami három fő tábla köré épül:

A products tábla csak az eszköz alapadatait tartalmazza: a nevét, a kategóriáját és a típusát. Itt nincsenek konkrét tulajdonságok, csak az, hogy „van egy ilyen termékünk”.

A properties tábla tartalmazza azokat a tulajdonságokat, amelyek egy eszközhöz tartozhatnak: gyártó, modell, teljesítmény, ár stb.

A values tábla köti össze a kettőt, és ide kerül a konkrét érték. Például: product = hangfal, property = teljesítmény, value = 120 W – ez egyetlen sor a values táblában. Ennek a megoldásnak az a nagy előnye, hogy nem kell minden tulajdonsághoz külön oszlopot felvenni, és bármikor lehet új tulajdonságot vagy új kategóriát hozzáadni anélkül, hogy a szerkezetet át kelljen alakítani. Az új termékek beillesztése is egyszerűbb, és a frontend is egységesen tudja kezelni az adatokat.

A setupokhoz külön tábla-rendszer tartozik. A setups tábla magát a setupot tárolja (pl. „Nappali házimozi”), és azt is, hogy melyik felhasználóhoz tartozik. Egy felhasználónak több setupja is lehet, ez egy klasszikus one-to-many kapcsolat. A setup\_devices tábla az adott setupban szereplő eszközöket tartalmazza, beleértve

azt is, hogy az eszköz hol helyezkedik el (pos\_x, pos\_y) és milyen irányba áll (rotation) – erre azért van szükség, mert a setupokat vizuálisan, egy munkaterületen jelenítjük meg.

Az eszközök közötti kapcsolatokat a setup\_connections tábla kezeli. Ez egy gráfszerű felépítést ad, amely megmondja, hogy melyik eszközből melyik másik eszközbe milyen porton keresztül és milyen kábellel megy a jel. A portokat és kábeleket külön tartjuk nyilván:

setup\_device\_ports – egy adott eszköz portjai,

port\_types – létező port típusok (HDMI, USB, RCA stb.),

cables\_info – kábelek típusa és tulajdonságai.

Ennek köszönhetően a rendszer nem csak azt tudja, hogy „A eszköz össze van kötve B eszközzel”, hanem azt is, hogy technikailag lehetséges-e a kapcsolat (pl. egy HDMI kimenetet nem lehet RCA bemenetbe dugni).

A felhasználói adatok két táblában vannak: az user[Auth] a bejelentkezéshez szükséges alap adatokat (név, email, jelszó hash, Google ID), az user\_more[Auth] pedig a kiegészítő profil adatokat (város, telefonszám, kor stb.) tárolja.

Összességében a végső szerkezet sokkal karbantarthatóbb és bővíthetőbb, mint az eredeti, sok különálló táblát használó modell. Új termék kategóriát ma már néhány új sor hozzáadásával be lehet illeszteni a rendszerbe, anélkül, hogy bármit át kellene szervezni vagy új táblát kellene létrehozni hozzá.

## 4.2 Táblák és kapcsolatok leírása

### 4.2 Táblák és kapcsolatok leírása

Ebben a részben részletesebben bemutatjuk a fő táblákat és azt, hogy ezek hogyan kapcsolódnak egymáshoz. A szerkezet úgy lett kialakítva, hogy minden tábla egyértelmű feladatot lásson el, és a kapcsolatokat logikusan, idegen kulcsokon (foreign key) keresztül jöjjenek létre.

Termékekhez kapcsolódó táblák

products – A rendszerben elérhető összes termék alap adatait tárolja. Egy sor egy konkrét eszközt jelent (pl. egy adott hangfal vagy egy adott PC alkatrész). Minden terméknek van egyedi azonosítója (id), neve, típusa és kategóriája.

properties – A termékekhez tartozó tulajdonság-fajtákat sorolja fel. Ez nem konkrét érték, hanem azt mondja meg, hogy milyen jellegű adat lehet egy termékhez kapcsolva (pl. „gyártó”, „modell”, „teljesítmény”, „ár”). Minden tulajdonságnak van saját azonosítója és neve.

values – Ez a tábla köti össze a productot a propertyvel, és ide kerül maga a konkrét érték. Egy sor azt mondja meg, hogy egy adott termékhez (products\_id) egy adott tulajdonság (properties\_id) milyen értéket vesz fel. Például: products\_id = 12, properties\_id = 3, value = "120W".

A három tábla együtt adja az EAV modellt. A kapcsolat many-to-many jellegű: egy termékhez több tulajdonság is tartozhat, és ugyanaz a tulajdonság több terméknél is használható.

Setupokhoz kapcsolódó táblák

setups – Ide kerülnek a felhasználó által létrehozott setupok. Minden setup egyetlen sor, és tartalmaz egy user\_id mezőt, ami megmondja, melyik felhasználóhoz tartozik. Innen jön a one-to-many kapcsolat: egy felhasználónak több setupja lehet, de egy setup csak egy felhasználóhoz tartozik.

setup\_devices – Egy setupban elhelyezett eszközöket írja le. Itt nem csak az szerepel, hogy milyen eszközről van szó (device\_id a productsra mutat), hanem az is, hogy a setupon belül hol helyezkedik el (pos\_x, pos\_y) és milyen irányba néz (rotation). A setup\_id mező köti össze a megfelelő setuppal. Itt is one-to-many a kapcsolat: egy setupban több eszköz is lehet.

setup\_connections – Az eszközök közötti összeköttetéseket tartalmazza. Egy sor mindig egy kapcsolatot ír le: melyik eszközből (from\_device\_id), melyik másik eszközbe (to\_device\_id), melyik porton keresztül és milyen kábellel megy a jel. Ez gyakorlatilag egy gráfot ír le a setupon belül.

Portok és kábelek

setup\_device\_ports – Egy adott eszköz portjait tárolja (pl. egy erősítőnek lehet 4 RCA bemenete és 2 HDMI bemenete). Minden port a saját eszközhöz tartozik, ezért itt is van egy device\_id kapcsolat a setup\_devicesra.

port\_types – A rendszerben létező port típusokat sorolja fel (HDMI, USB, RCA, optikai stb.). Ez egy szótár-jellegű tábla, hogy mindenhol egységesen tudjunk hivatkozni egy port típusra.

cables\_info – A kábelek adatai: típus, hossz, milyen porthoz illik. Ehhez kapcsolódik a setup\_connections, hogy egyértelmű legyen, milyen kábellel valósul meg az összeköttetés.

Ennek a felosztásnak köszönhetően nem lehet érvénytelen kapcsolatot létrehozni: a port típusa és a kábel típusa ellenőrzött, így például HDMI kimenetet nem lehet RCA bemenetbe kötni.

Felhasználói táblák

user[Auth] – A bejelentkezéshez szükséges alap adatokat tárolja: felhasználónév, név, email, jelszó hash, Google ID. Minden felhasználónak van egy egyedi ID mezője, amire a rendszer többi része hivatkozik.

user\_more[Auth] – A felhasználó kiegészítő profil adatait tartalmazza (város, kor, telefonszám stb.). Itt egy user\_id köti össze az user[Auth] táblával, így minden felhasználóhoz egy ilyen kiegészítő sor tartozik. Ez one-to-one kapcsolat.

user\_register\_code[Auth] és user\_password\_reset[Auth] – Az email alapú regisztrációs és jelszó visszaállítási kódokat tárolja. Mindkettő tartalmaz egy lejáratí időt és egy „használt” jelzőt, így a kódok csak egyszer és meghatározott időn belül érvényesek.

Kapcsolatok összefoglalása

user[Auth] → setups – egy felhasználónak több setupja lehet (one-to-many).

user[Auth] → user\_more[Auth] – egy felhasználóhoz egy kiegészítő profil tartozik (one-to-one).

setups → setup\_devices – egy setupban több eszköz lehet (one-to-many).

setup\_devices → products – minden setup-eszköz egy konkrét termékhez kapcsolódik (many-to-one).

products ↔ properties – values táblán keresztül (many-to-many, EAV).

setup\_connections → setup\_devices – két eszközt köt össze (gráf jellegű kapcsolat).

setup\_device\_ports → port\_types – minden port egy típusra hivatkozik (many-to-one).

setup\_connections → cables\_info – minden kapcsolat egy kábellel valósul meg (many-to-one).

Ez az elrendezés biztosítja, hogy az adatok között konzisztens hivatkozások legyenek, és hogy a setupokat akár vizuálisan is pontosan vissza lehessen állítani: tudjuk, milyen eszközök vannak benne, hol helyezkednek el, milyen porttal és kábellel vannak összekötve, és melyik felhasználóhoz tartozik az egész.

## 5. Backend Megvalósítás (kód részlettel)

A backend megvalósítása Node.js + Express környezetben történt, REST alapú API architektúrával. Az alkalmazás fő feladata a különböző termékkategóriák kezelése, dinamikus lekérdezések biztosítása, valamint a felhasználói adatok és konfigurációk kezelése.

Az adatkezelés Supabase segítségével történik, amely PostgreSQL alapú backend szolgáltatást biztosít.

#### A backend fő rétegei:

- **Controller** réteg – HTTP kérések kezelése
- **Route** réteg – végpontok definiálása
- **Service** réteg – üzleti logika (pl. productCatalog.service)
- **Adatbázis** réteg – Supabase

### Termékkezelés (Products modul)

- A termékek lekérdezése egységes módon történik a productCatalog.service segítségével.

#### Funkciók:

- Termékek listázása (/api/products)
- Márkák lekérdezése (/api/products/brands)
- Szűrés keresési kulcsszó alapján

#### Működés:

A controller a query paraméterekből dolgozik:

- q → keresés
- limit → maximum elemszám

```
const { listProducts, listBrands, clampLimit } = require("../services/products/productCatalog.service");

async function list(req, res) : Promise<void> { Show usages HazafiLevente +1
  const q : string = (req.query.q ?? "").trim();
  const limit : number | any = clampLimit(req.query.limit, { fallback: 200 });

  try {
    const items : any[] = await listProducts({ options: { q, limit, category: "all" } });
    res.json({ items });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
}
```

## 11. Ábra: Kontroller Kód

A clampLimit biztosítja, hogy a lekérdezés ne terhelje túl a rendszert.

## Kategória-specifikus endpointok

A rendszer külön endpointokat biztosít különböző termékkategóriákhoz:

- Autók (/api/cars)
- Számítógépek (/api/computers)
- Hangszerek (/api/instruments)
- Házimozi eszközök (/api/hometheaters)

```
try {
  const items :any [] = await listProducts( options: {limit, category: "car" });

  res.json({ items });
}
```

## 12. Ábra: Endpointok

Ez a megközelítés lehetővé teszi:

- egységes adatkezelést
- könnyű bővíthetőséget új kategóriákkal

## Rangrendszer (Ranks modul)

A rendszer tartalmaz egy felhasználói szint- és pontkezelő rendszert.

### Funkciók:

- Felhasználó aktuális szintjének lekérdezése
- Pontok számítása
- Következő sinthez szükséges pont meghatározása

### Működés:

A rendszer két táblából dolgozik:

- user\_level
- level\_ranks

A progress érték számítása:

```
const progress :number = max > min ? Math.max( values: 0, Math.min( values: 1, (points - min) / (max - min))) : 0;
```

## 13. Ábra: Progress érték

Az endpoint autentikációhoz kötött (verifyUser middleware).

## Home Theater modul

Ez a rendszer egyik legösszetettebb része.

### Fő funkciók:

- Eszközkatálogos kezelése
- Házimozsi konfigurációk (build-ek)
- Eszközök elhelyezése térben
- Kapcsolatok kezelése eszközök között

### Build kezelés

Lehetővé teszi több konfiguráció létrehozását egy szobán belül.

Funkciók:

- Build létrehozása / módosítása
- Build lekérdezése
- Build törlése

```
router.post( path: "/build", verifyUser, ctrl.saveBuild);
```

### 14. Ábra: Build kód

### Eszközök kezelése

Az eszközök pozícióval és típussal rendelkeznek:

- pos\_x, pos\_y
- rotation
- role

A rendszer automatikusan szinkronizálja az adatbázist:

```
await syncDevices(build.id, devices, layout);
```

### 15. Ábra: Szinkronizálás

## Kapcsolatok kezelése

A felhasználó összekötheti az eszközöket:

- pl. erősítő → hangfal

```
router.get(path: "[:setupId/connections", verifyUser, ctrl.listConnections);
router.post(path: "/connection", verifyUser, ctrl.createConnection);
router.delete(path: "/connection/:id", verifyUser, ctrl.deleteConnection);
```

## 16. Ábra: Kapcsolatok

## Meta adatok és dinamikus szűrés

A backend dinamikusan generál szűrőket az adatok alapján.

- gyártók listája
- táblák
- numerikus mezők (pl. ár)
- boolean mezők
- enum mezők

A rendszer automatikusan felismeri:

- szám típusokat
- logikai értékeket
- kategóriákat

```
if (isNumericLike(clean)) {
    numbers.push(key);
    continue;
}
```

## 17. Ábra: Meta adatok és dinamikus szűrés

Ez lehetővé teszi a frontend számára, hogy:

- dinamikus filter UI-t építsen
- ne kelljen fix mezőket definiálni

## Egyedi item lekérdezés

Az /api/items/:table/:id endpoint dinamikusan képes lekérni adatokat bármely táblából.

## Biztonság:

- csak valid table name engedélyezett
- input validáció regex-szel

```
if (!/^[a-z0-9_]+$/.test(table)) {  
  return res.status(400).json({ error: "Invalid table name" });  
}
```

## 18. Ábra: Biztonság és szűrés

### Képek kezelése

#### Image map endpoint

A rendszer automatikusan feltérképezi a képeket:

- mappa struktúra alapján
- táblák és ID-k szerint

#### Képletöltő script

Automatikus képletöltés több lépésben:

- Bing image search scraping
- duplikáció szűrés (hash alapján)
- minimum méret ellenőrzés

#### Fő fájlok:

- download-images.mjs
- download-images-pass2.mjs
- download-images-pass3.mjs

#### Fő jellemzők:

- retry mechanizmus
- timeout kezelés
- több keresési stratégia

## Biztonság

A rendszer több helyen védi az adatokat:

- verifyUser middleware
- user ownership ellenőrzés (assertSetupOwnedByUser)
- input validáció

## 5.1 REST API végpontok

Az alkalmazás REST API-t használ: a kliens HTTP kéréseket küld (GET, POST, PATCH, PUT, DELETE), a válaszok többnyire JSON-ben érkeznek. Az összes backend végpont útvonala /api alatt kezdődik. Fejlesztés közben az Angular proxy miatt ezek gyakorlatilag ugyanazon a kiszolgálón érhetők el (pl. /api/auth/login).

Egészség ellenőrzés: GET /api/test-live – egyszerű válasz, hogy a backend fut-e.

Termékek, metaadatok, szűrés

/api/products – összesített terméklista (GET /), gyártók (GET /brands).

/api/computers, /api/cars, /api/instruments, /api/home-theater – adott kategória termékeinek listázása (és a házimozsi ágon build / eszköz / kapcsolat kezelés is).

/api/meta – dinamikus szűrő-metaadatok táblánként (autó, PC, házimozsi, hangszer).

/api/items – egyedi rekord lekérése tábla és id alapján (GET /:table/:id), illetve lista / keresés (ezekhez bejelentkezés kell).

Bejelentkezés és felhasználó

/api/auth – regisztráció, email kód kérése / ellenőrzése, belépés, kijelentkezés, jelszó reset kérés / megerősítés, Google OAuth belépés; GET /me – aktuális felhasználó (csak bejelentkezve).

/api/profile – saját profil lekérése és módosítása, jelszócsere.

/api/users – felhasználók listázása (védett), illetve nyilvános / opcionálisan autentikált profilnézet felhasználónév vagy id alapján.

Setupok (MySetup)

Minden /api/setup/... végpont mögött kötelező bejelentkezés van.

Ide tartozik:

szoba / setup létrehozás, listázás, módosítás, törlés, lista-kezelés (kedvencek / csoportok szerinti rendezés),

PC build, autó, hangszer, hálózat hozzáadása és adatainak lekérése,  
eszközök mentése, átnevezése, pozíciója, törlése,  
eszközök közötti kapcsolatok lekérése, létrehozása, törlése,  
„gyerek” setupok és egyéb opciók lekérése.

Ranglista, értesítések, üzenetek

/api/ranks/me – bejelentett felhasználó pontjai / szintje.

GET /api/leaderboard – ranglista.

/api/bell – csengő / értesítések, beszélgetések összefoglalója, olvasás jelölése.

/api/messages – szintén csak bejelentkezve: beszélgetés indítása, üzenet küldése, panelek,  
beszélgetés törlése / szerkesztése, AI chatszálak (/ai/...), illetve kapcsolatok (mute / block /  
disable).

Adminisztráció

Mind /api/admin, mind /api/admin/products jogosultsághoz kötött (admin / admin+ /  
owner szintektől függően):

statisztikák, aktív felhasználók,

reportok listázása és törlése,

felhasználók listázása, módosítása, tiltás / feloldás, egy felhasználó setupjainak áttekintése,  
rendszerüzenetek (admin+),

szerver naplók olvasása / törlése (owner),

admin termékkezelés: típus sablonok, táblánként CRUD (GET/POST/PATCH/DELETE  
dinamikus útvonalakon).

Egyéb

/api/public – korlátozott, nyilvános tábla-adat (pl. táblanév szerint).

GET /api/images/map – képek feltérképezése (image map).

POST /api/ai/ask – fő AI kérdés (bejelentkezés szükséges).

Megjegyzés: Sok végpontnál a pontos HTTP metódus és a „ki férhet hozzá” (mindenki /  
bejelentkezett / admin) a route fájlokban és a middleware-ekben van rögzítve; a  
dokumentációban elég ennek a csoportosításnak megfelelni, részletes referenciát pedig  
OpenAPI / táblázat formában lehet később kitölteni.

## 5.2 Jogosultságkezelés

A jogosultságkezelést rangokra bontottuk, jelenleg négy rang van: User, Admin, Admin+, Owner. Az alap mindenképpen a User, ezt kapja meg minden új regisztrált felhasználó. Ezen a szinten lehet keresni, AI-t használni, setupokat létrehozni, módosítani, törölni, üzenetet küldeni és minden olyan funkciót használni, ami a hétköznapi működéshez kell.

Az Admin rang egy picivel többet tud, mint a User. Az adminok kezelik a beérkező report-okat, és ők moderálják a felhasználókat. Tudnak bannolni is, de a fontosabb műveletekhez már nagyobb rang kell.

Az Admin+ rang a programozók rangja. Ők hozzáférnek az SQL táblákhoz, tudnak felhasználókat bannolni és feloldani, illetve mindent megtehetnek, amit egy sima Admin.

Az Owner a legmagasabb rang, ezt nem lehet sem elvenni, sem átadni, sem módosítani. Az ownert a kód kifejezetten védi, így admin felületről sem lehet lefokozni.

A rangokat a .env fájlban tároljuk, felhasználói azonosítók szerint, vesszővel elválasztva (OWNERS, ADMINS\_PLUS, ADMINS, BANNED\_USERS). A szerver induláskor ezeket beolvassa, és a rang lekérdezése egy egyszerű halmaz-ellenőrzés a resolveRole() függvénnyel. Azért nem adatbázisba tettük, mert így egy téves SQL módosítás sem tud belenyúlni a jogokba, és a tulajdonosok listája sem szivárogo ki egy esetleges adatbázis-lekérdezésből.

A bejelentkezés JWT alapon működik. Sikeres login után a backend kiállít egy token-t, amit egy auth\_token nevű HttpOnly süti-be rak, így a JavaScript a kliens oldalon nem fér hozzá. A token alapból 12 órán át érvényes, a Remember me opcióval 25 napig. A frontenden egy interceptor minden kérésre automatikusan ráteszi a sütit, így a hitelesítés transzparensen működik.

A védett végpontokat middleware-ek őrzik. A verifyUser minden bejelentkezést igénylő route előtt lefut, leellenőrzi a token-t, és beállítja a req.user objektumot. A verifyAdmin, verifyAdminPlus és verifyOwner ezen felül még a rangot is ellenőrzi, és csak megfelelő szint esetén enged tovább. Fontos, hogy a rangot nem a tokenből vesszük ki, hanem mindig frissen kérdezzük le, így ha valakinek menet közben módosul a rangja, nem kell újra bejelentkeznie.

A rangok kiosztásánál is van egy belső szabály: senki sem tud magánál magasabb rangot adni másnak, és csak nála alacsonyabb rangú felhasználót tud módosítani. Ezt a canAssignRole() függvény és az admin route-on belüli rank() ellenőrzés biztosítja.

Az ownership ellenőrzés külön réteg: a saját setupokat csak a tulajdonosuk tudja módosítani, mert a backend mindig összeveti a req.user.id-t az adatbázisban tárolt user\_id mezővel. Adminok ezt az admin route-okon keresztül megkerülhetik.

Frontend oldalon a kapott rang alapján bizonyos menüpontok (pl. az Admin gomb a dock menüben) csak megfelelő szint esetén jelennek meg, de ez csak a felhasználói élmény miatt van így, a tényleges védelmet mindig a backend végzi.

Érdemes különbséget tenni a rang és a szint között. A rang szabályozza, hogy ki mit tehet meg az oldalon, a szint viszont csak egy gamifikációs elem: a felhasználó aktivitása alapján kapott pontok szerint emelkedik 1-től 10-ig, és kizárólag a profilon és a ranglistán látszik – jogosultságot nem ad.

A szintek tartományai: [1: 0–250; 2: 251–500; 3: 501–1000; 4: 1001–1500; 5: 1501–2000; 6: 2001–3000; 7: 3001–4000; 8: 4001–5000; 9: 5001–10 000; 10: 10 001–20 000].

## 6. Frontend megvalósítás (kód részlettel)

A projekt frontend része Angular keretrendszer használatával készült, komponens alapú felépítésben. A fejlesztés során fontos szempont volt, hogy a felhasználói felület átlátható, modern és könnyen kezelhető legyen. Az alkalmazás több különálló komponensből épül fel, amelyek külön feladatokat látnak el, például a termékek megjelenítését, a keresési és szűrési funkciókat, a részletes termékadatok megjelenítését, valamint a felhasználói rangrendszer és ranglista kezelését.

A főoldalon a termékek kártya formájában jelennek meg. A terméklista dinamikusan töltődik be az adatbázisból, és Angular ciklus segítségével kerül megjelenítésre.

```
1 <div class="product-grid" *ngIf="pagedProducts.length > 0">
2   <app-product
3     *ngFor="let p of pagedProducts"
4     [product]="p"
5     (openProduct)="onOpenProduct($event)">
6   </app-product>
7 </div>
```

### 19. Ábra: NgFor kód

Az Angular `*ngFor` direktíva egy strukturális direktíva, amely lehetővé teszi, hogy egy tömb elemein végigiteráljunk, és minden elemhez automatikusan létrejöjjön egy HTML elem vagy komponens.

A rendszer működése során a backendből érkező terméklista egy tömbben kerül eltárolásra (pagedProducts). Ez a tömb már egy szűrt és lapozott lista, amelyet a frontend állít elő.

Amikor az Angular rendereli a komponenst, a `*ngFor` végigmegy a pagedProducts elemein, és minden egyes termékhez létrehoz egy külön `<app-product>` komponenst.

Ez a megoldás biztosítja:

- *a dinamikus megjelenítést*
- *az új adatok automatikus kirajzolását*
- *a komponens alapú újrafelhasználhatóságot*

## Lapozási Rendszer

A lapozási rendszer célja a teljesítmény optimalizálása. Mivel az adatbázis közel 1502 terméket tartalmaz, nem lenne hatékony az összes elemet egyszerre megjeleníteni.

A frontend ezért egy szeletelési (slice) technikát alkalmaz.

```
private updatePaged() { Show usages  Hazafi Levente
  const total = this.filteredProducts.length;
  this.totalPages = Math.max(1, Math.ceil(total / this.PAGE_SIZE));

  const start = (this.page - 1) * this.PAGE_SIZE;
  this.pagedProducts = this.filteredProducts.slice(start, start + this.PAGE_SIZE);
}

prevPage() { Show usages  Hazafi Levente +1
  if (this.page > 1) {
    this.page--;
    this.updatePaged();
    window.scrollTo({ top: 0, behavior: 'smooth' });
  }
}

nextPage() { Show usages  Hazafi Levente +1
  if (this.page < this.totalPages) {
    this.page++;
    this.updatePaged();
    window.scrollTo({ top: 0, behavior: 'smooth' });
  }
}

/* -----
```

## 20. Ábra: Lapozási rendszer

A működés lépései:

1. **A teljes lista a *filteredProducts* tömbben található**
2. **A *current page* (*page* változó) határozza meg az aktuális oldalt**
3. **A *start index* kiszámítása:**  
 $\text{start} = (\text{page} - 1) * \text{PAGE\_SIZE}$
4. **A megjelenített elemek:**  
`filteredProducts.slice(start, start + PAGE_SIZE)`

Ez a megoldás:

- **csökkenti a DOM terhelést**
- **gyorsítja az oldal betöltését**
- **stabil működést biztosít nagy adatmennyiségnél**

A rendszer a terméklistát kisebb oldalakra bontja, így egyszerre csak egy meghatározott számú elem jelenik meg. Ez javítja a teljesítményt, különösen nagyobb adatállomány esetén.

## Keresési és szűrési rendszer

A termékek közötti keresést és szűrést egy külön keresősáv és szűrő rendszer valósítja meg. A felhasználó több szempont alapján kereshet, például gyártó, modell vagy ár szerint.

A keresési rendszer egy központi szűrőszolgáltatáson keresztül működik (*filtersService*).

```
searchButton(): void { Show usages AWS2359
  const filters = {
    term: (this.term || '').trim(),
    manufacturer: (this.manufacturer || '').trim(),
    priceMin: this.priceMin,
    priceMax: this.priceMax,
    sort: (this.sort || '').trim()
  };

  console.log('SEARCHBAR -> setFilters()', filters);

  this.filtersService.setSearch(filters as SearchFilters);
}
```

### 21. Ábra: Keresési és szűrési rendszer

A `searchButton()` függvény összegyűjti a felhasználó által megadott adatokat:

- *keresési szöveg*
- *gyártó*
- *minimum ár*
- *maximum ár*
- *rendezési mód*

Ezeket egy objektumba csomagolja, majd átadja a szolgáltatásnak:

```
this.filtersService.setSearch(filters)
```

A szolgáltatás egy `BehaviorSubject` segítségével tárolja az aktuális szűrőállapotot, így minden komponens automatikusan frissül, ha a felhasználó változtat a feltételeken.

Ez reaktív működést biztosít:

- nincs szükség manuális frissítésre
- minden változás azonnal megjelenik

A fenti függvény a felhasználó által megadott keresési feltételeket továbbítja a szűrőszolgáltatás felé, amely ez alapján frissíti a megjelenített terméklistát.

## Kategória alapú szűrés

A kategória szűrés célja, hogy a felhasználó gyorsan válthasson különböző terméktípusok között.

```

toggle(key: CategoryKey) {
  const cur = this.filtersService.current.activeCategory;
  const next: CategoryKey = (cur === key) ? 'all' : key;

  this.active.car = false;
  this.active.computer = false;
  this.active.ht = false;
  this.active.instrument = false;

  if (next === 'car') this.active.car = true;
  if (next === 'computer') this.active.computer = true;
  if (next === 'ht') this.active.ht = true;
  if (next === 'instrument') this.active.instrument = true;

  this.filtersService.setActiveCategory(next);
}

```

## 22. Ábra: Kategória alapú szűrés

A toggle () függvény ellenőrzi az aktuális kategóriát:

- *ha ugyanarra kattint, → visszaáll "all"*
- *ha másra → aktiválja az adott kategóriát*

Ezután:

this.filtersService.setActiveCategory (next)

Ez biztosítja:

- globális állapotkezelést
- minden komponens szinkronizálását

Ez a kódrészlet azt a logikát mutatja be, amely lehetővé teszi, hogy a felhasználó különböző kategóriák között váltson, például autók, számítógépek, hangszerek vagy házimozzi eszközök között.

## Oldalsó termék panel

A termékkártyákra kattintva egy oldalsó panel jelenik meg, amely az adott termék részletes adatait tartalmazza.

```

ngOnChanges(): void { no usages  👤 HazafiLevente
  const table = this.getTable(this.product);
  const id = this.getId(this.product);
  if (!table || id == null) return;

  const key = `${table}::${id}`;
  if (key === this.lastKey) return;

  this.lastKey = key;

  this.lockScroll();
  this.fetchDetails();
}

```

## 23. Ábra: Oldalsó termék panel

Minden alkalommal, amikor a kiválasztott termék változik:

- *új azonosító generálódik*
- *a rendszer ellenőrzi, hogy történt-e változás*
- *ha igen → betölti az új adatokat*

Ez a megoldás biztosítja:

- *hogyan ne töltse újra feleslegesen az adatokat*
- *gyorsabb felhasználói élményt*

Ez a megoldás lehetővé teszi, hogy a felhasználó gyorsan megtekinthesse a termék részleteit anélkül, hogy el kellene hagynia az aktuális oldalt.

## Felhasználói rangrendszer

A rendszer tartalmaz egy felhasználói rangrendszert is, amely a felhasználó aktivitása alapján jeleníti meg az aktuális szintet.

A frontend a rankService segítségével lekéri a felhasználó pontjait:

```
ngOnInit() { no usages HazafiLevente
  this.rank.me().subscribe({
    next: (d) => { this.data = d; this.loading = false; },
    error: () => { this.loading = false; }
  });
}
```

## 24. Ábra: Felhasználói Rangrendszer

`this.rank.me().subscribe(...)`

A rendszer:

- *kiszámolja a szintet*
- *vizuálisan megjeleníti*
- *frissíti a UI-t*

Ez motivációs rendszert biztosít a felhasználóknak.

A rangrendszer pontok alapján számolja a felhasználók szintjét, amely vizuálisan is megjelenik a profil oldalon.

## Spotlight termékajánló

A főoldal jobb oldalán egy Spotlight termékajánló található, amely véletlenszerűen vagy automatikusan váltakozó módon jelenít meg termékeket az adatbázisból. Ennek a funkciónak az a célja, hogy a felhasználó keresés nélkül is találkozzon különböző termékekkel, így az oldal interaktívabb és látványosabb lesz.

A Spotlight komponens a főoldalon külön blokkban jelenik meg, és a termék listából érkező szűrt elemeket használja fel. Ez azt jelenti, hogy a jobb oldali ajánló mindig az aktuálisan elérhető termékhalmazból dolgozik.

```

<div class="right desktop-only">
  <app-quick-stats
    [total]="totalAll"
    [pool]="featuredPool.length"
    [chips]="allCategories">
  </app-quick-stats>

  <app-featured-spotlight
    [products]="featuredPool"
    (openProduct)="onOpenProduct($event)">
  </app-featured-spotlight>
</div>
</div>

```

## 25. Ábra: Spotlight és FeaturedPool kód

A fenti kódrészlet azt mutatja meg, hogy a Spotlight ajánló az oldal jobb oldalán kapott helyet. A komponens a **featuredPool** tömbből kapja meg a megjelenítendő termékeket.

```

onProductsChanged(list: AnyProduct[]) { Show usages HazafiLevente
  this.featuredPool = list || [];
}

```

Ez a függvény biztosítja, hogy a Spotlight ajánló mindig a termék listából érkező aktuális elemekkel dolgozzon.

## Navigációs dock menü

Az alkalmazás tartalmaz egy alsó navigációs menüt is, amely gyors elérést biztosít a legfontosabb oldalakhoz. A menürendszer célja az volt, hogy a felhasználó a legfontosabb funkciók között egyetlen kattintással tudjon váltani, különösen kisebb képernyőn vagy hosszabb görgetés után.

A dock menüben a felhasználó közvetlenül elérheti a főbb nézeteket, például a főoldalt, a kedvencek listáját, a setup oldalt, a profilt, az üzeneteket, a beállításokat, illetve jogosultság esetén az admin felületet is. Ez a megoldás gyorsabb és kényelmesebb navigációt biztosít, mint a hagyományos felső menürendszer.

A dock komponens a frontend különálló navigációs egységként került be az alkalmazásba, és a komponens alapú felépítés részeként lett integrálva.

```

import { DockComponent, DockItemData } from '../Shared/Dock/dock.component';

```

## 26. Ábra: Dock kód

A fenti kódrészlet azt mutatja, hogy a dock menü külön Angular komponensként került beépítésre a rendszerbe.

Mivel a navigáció külön komponensben lett megvalósítva, a menü logikája és megjelenítése elkülönül az oldal többi részétől. Ez könnyebbé teszi a továbbfejlesztést és az új menüpontok hozzáadását.

## Animációk és vizuális effektek

A frontend fejlesztés során fontos szempont volt, hogy az oldal ne csak funkcionálisan működjön megfelelően, hanem vizuálisan is modern és interaktív legyen. Ennek érdekében több animációs és effektum került beépítésre a rendszerbe.

Az egyik ilyen megoldás a kattintási effekt, amely vizuálisan visszajelzi a felhasználó műveleteit. Ez növeli az oldal interaktivitását, és modernebb hatást kelt.

```
onClick(e: MouseEvent): void { Show usages HazafiLevente

    if (!this.enabled) return;
    if (typeof window === 'undefined') return;
    console.log('CLICK SPARK fired');

    const rect = this.canvasRef.nativeElement.getBoundingClientRect();
    const x = e.clientX - rect.left;
    const y = e.clientY - rect.top;

    const now = performance.now();

    for (let i = 0; i < this.sparkCount; i++) {
        this.sparks.push({
            x,
            y,
            angle: (2 * Math.PI * i) / this.sparkCount,
            startTime: now
        });
    }
}
```

## 27. Ábra: Animációk és vizuális effektek

A fenti kódrészlet funkció lényege, hogy minden kattintáskor több kis animált vonal vagy “szikra” jelenik meg a kattintás helyén.

Az animációs rendszer másik része a háttér effekt, amely shader alapú megoldással folyamatos mozgást és különleges vizuális hatást biztosít a háttérben.

```
private loop = () => { Show usages  HazafiLevente
  if (!this.renderer || !this.program || !this.mesh) return;

  const t = ((performance.now() - this.start) / 1000) * this.speed;

  this.program.uniforms['uTime'].value = t;
  this.program.uniforms['uHueShift'].value = this.hueShift;
  this.program.uniforms['uNoise'].value = this.noiseIntensity;
  this.program.uniforms['uScan'].value = this.scanlineIntensity;
  this.program.uniforms['uScanFreq'].value = this.scanlineFrequency;
  this.program.uniforms['uWarp'].value = this.warpAmount;

  this.renderer.render({ scene: this.mesh });

  this.raf = requestAnimationFrame(this.loop);
};
}
```

## 28. Ábra: Háttér kód

Ez a kódrészlet a komponens egy folyamatosan frissülő háttér animációt rajzol ki, amely dinamikusabbá teszi a felhasználói felületet.

A vizuális elemek közé tartozik továbbá az elektromos keret effektus, az animált számláló, valamint a képgalériában használt mozgó kártyás megoldás is. Ezek az elemek nemcsak látványosabbá teszik az oldalt, hanem segítenek kiemelni bizonyos fontos információkat, például a felhasználói rangot vagy a statisztikai adatokat.

## MySetup

A saját feladatköröm(Egri Roland Imre) a **MySetup** funkció létrehozása, megtervezése, működtetése és tesztelése volt.

A **MySetup** oldal célja, hogy lehetőséget biztosítson saját setupok megtervezésére, tárolására, valamint a különböző lehetőségek összehasonlítására.

A fejlesztés első teszt fázisában HTML, CSS és JavaScript technológiákat használtunk. Ebben a szakaszban a **MySetup** alapja JavaScript nyelven készült el. Később a projekt Angular alapokra került

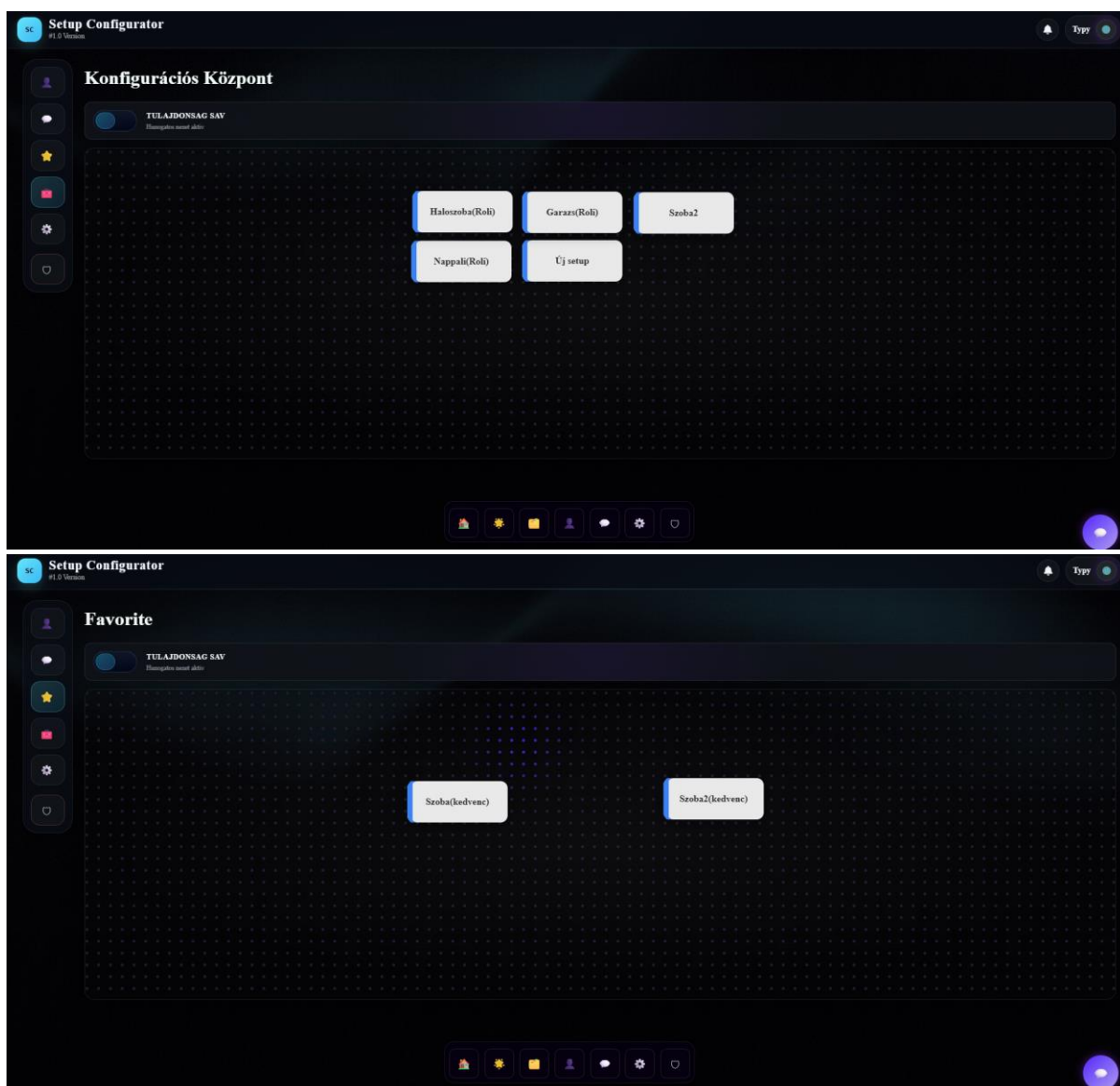
át, ezért az oldalt komponensekre kellett bontani, és a fejlesztés TypeScript, HTML és CSS használatával folytatódott.

Jelenleg a **MySetup** több külön komponensblokkból áll. Egy komponensblokk egy TypeScript-, egy HTML- és egy CSS-fájlból épül fel.

A tiszta kód elveinek megfelelően a fájlnevek rövidek, ugyanakkor egyértelműen utalnak a bennük található tartalomra.

A **MySetup** oldal dizájnya egy dot-grid megoldásra épül. Amikor a munkaterületen a kurzor vagy egy setup elem mozgatásra kerül, a háttér annak környezetében egy különleges effekt segítségével változik meg.

A következő képen is látható, hogy a **MySetup** oldal két fő részből áll: egy **Favorite** és egy **Konfigurációs központ** nézetből.



**29. Ábra: Konfigurációs központ, Kártyák**

Mind a **Favorite**, mind a setup konfigurációs oldalon lehetőség van műveletek végrehajtására a setup kártyákkal.

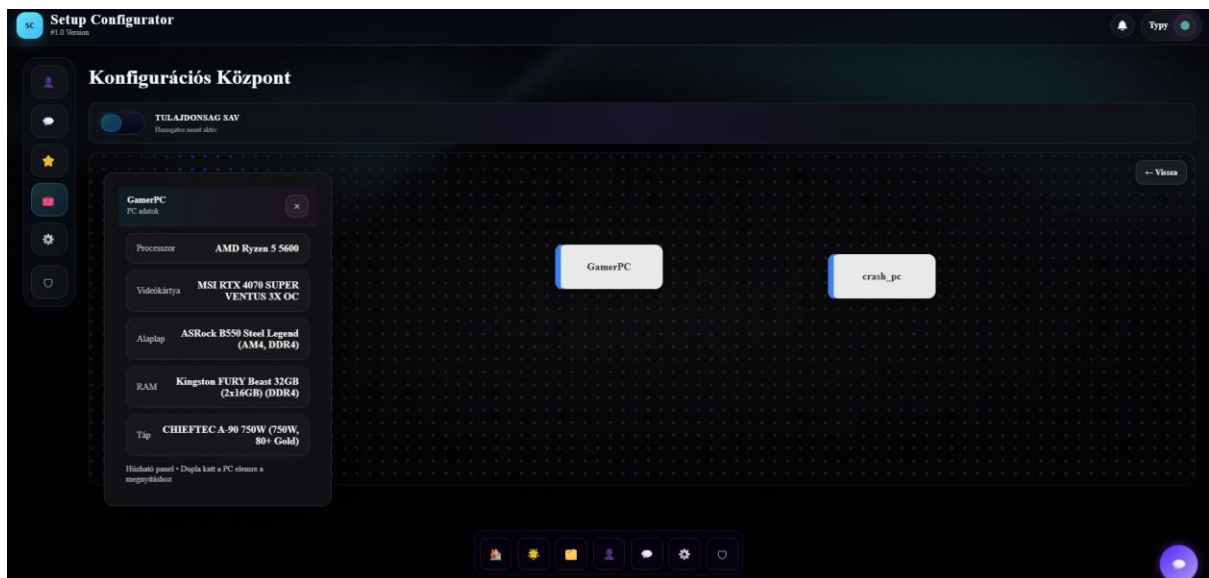
## MySetup oldalon navigálás/műveletek leírása

Új setup létrehozásához a dot-grid felületen jobb kattintással előhívható egy új eszköz hozzáadása gomb. Erre bal kattintással létrejön egy új setup.

Egy létrejött setup kártyára dupla bal kattintás után, megjelennek a setupban lévő eszközök, ha vannak. Ezen a felületen lehet hozzáadni egy új eszközt ahhoz a setuphoz, amelyikbe beléptünk a dupla bal kattintással.

Ehhez a dot-grid felületén kell egy jobb kattintás, ami után felugrik az új eszköz hozzáadása gomb, erre bal kattintással előjön egy menü, amin szerepel egy Számítógép gomb, egy Autók gomb, egy Házimozi gomb, egy Hangszerek gomb, és egy Egyéb gomb, amivel ki lehet választani milyen típusú elemet szeretnénk hozzáadni, a kiválasztott típusra bal kattintás, végig megyünk az adott típus hozzáadásán, tehát kiválasztunk egy konkrét elemet vagy elemeket például pc-k esetén, és hozzáadás gomb után látjuk is a setuphoz adott új eszközünket.

Egy már setuphoz adott eszközre dupla bal kattintás után, láthatjuk az adott eszköz paramétereit, tulajdonságlapját.



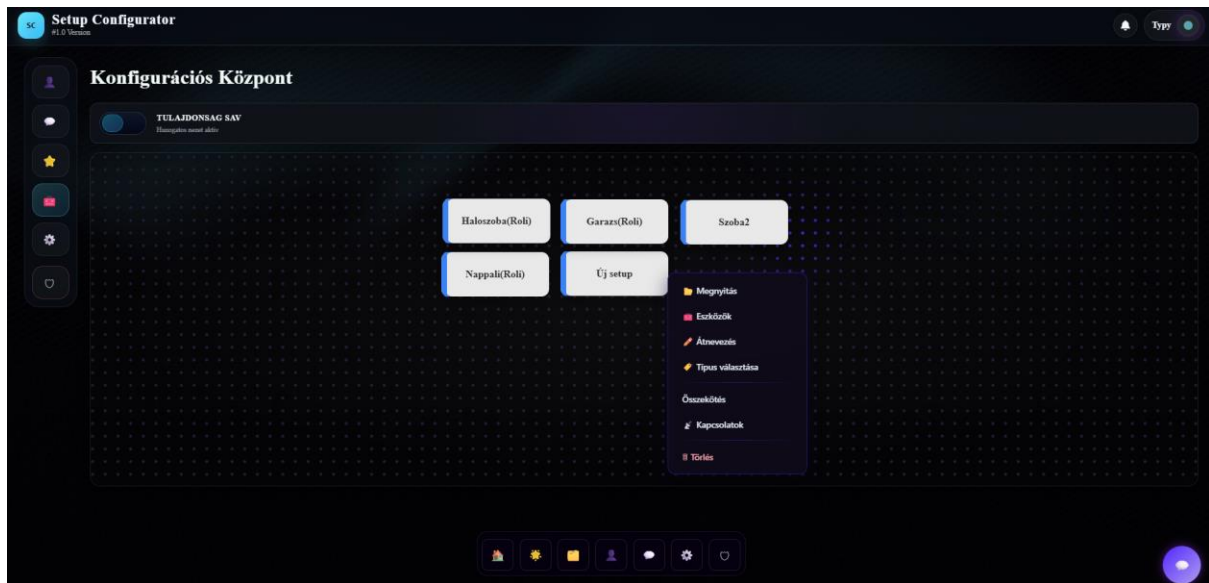
### 30. Ábra: Eszköz tulajdonságlap

#### Példa egy eszköz tulajdonság lapjára (dupla kattintás a kártyára)

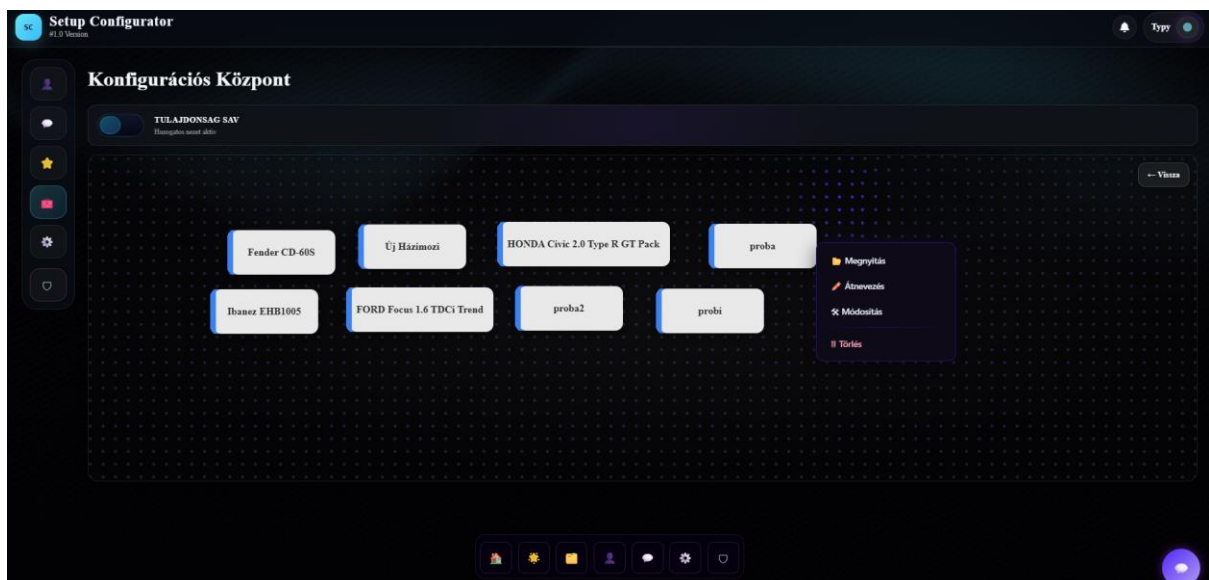
Egy már létrehozott setup kártyán jobb kattintás után megjelenik egy navigációs menü a kiválasztott kártya mellett. Ebben a menüben lehetőség van bal klikk után **Megnyításra**, ami megnyitja a setupot és az eszközök listázása mellett, átvezet az eszközök hozzáadásának helyére, tehát az adott setup belsejébe, **Eszközök** gomb a setupban lévő eszközöket jeleníti meg, **Átnevezés** gombbal a setup kártyát, ezzel magát a setupot lehet átnevezni, **Összekötés** gombbal két különböző de azonos típusú eszközt lehet összekötni, **Kapcsolatok** gomb az adott setupban mutatja meg azt, hogy vannak vagy nincsenek összekötött elemek a setupban, **Törlés** gomb az adott setupot törli, a benne lévő elemekkel együtt.

Egy már setuphoz hozzáadott eszközre jobb kattintás után szintén felugrik egy műveleteket tartalmazó menü, amiben van egy **Megnyitás** gomb ami előhossa az adott eszköz tulajdonságlapját,

egy **Átnevezés** gomb amivel egy adott eszközt átnevezhetjük, egy **Módosítás** gomb aminek segítségével módosíthatjuk az eszközünket, ha az egy házimozi típusú eszköz, egy **Törlés** gomb amivel egy adott eszközt törölhetünk.



31. Ábra: Setup menü

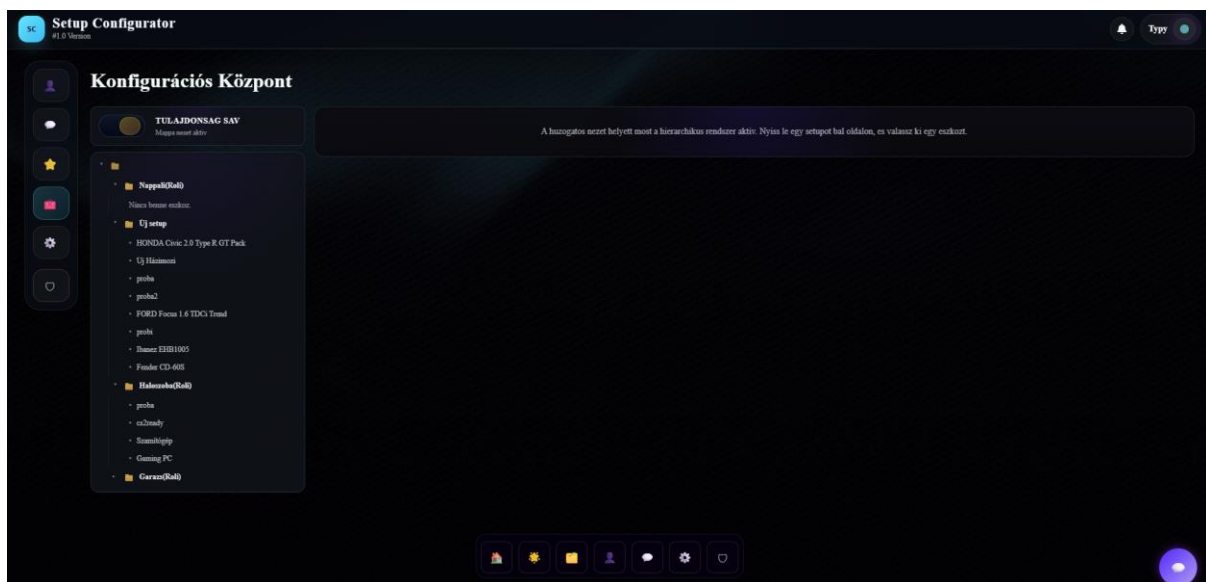


32. Ábra: Setup jobbkattintás

Bármely két setup összekapcsolható. Ehhez az egyik setup kártyán jobb kattintással meg kell nyitni a mellette megjelenő menüt, majd ki kell választani az összekötés lehetőségét. Ezután egy tetszőleges meglévő setup kiválasztható, és a kapcsolat létrejön.

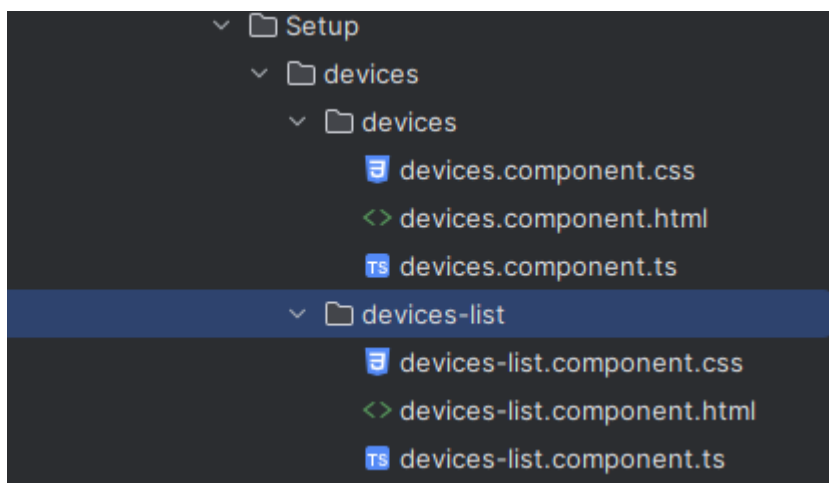
A rendszer a sikeres összekapcsolást vizuálisan is jelzi: a két setup között egy jól látható vonal jelenik meg.

Tartozik a konfigurációs központ funkcióihoz egy tulajdonság sáv, ami főként a mobilos nézet miatt lett hozzáadva, hogy ennek a segítségével, mobilon egy egyszerűbb, tömörebb, letisztult nézete legyen a setupoknak, ezt bekapcsolni a tulajdonság sáv felirat alatti csúszkára bal kattintással tudjuk megtenni.

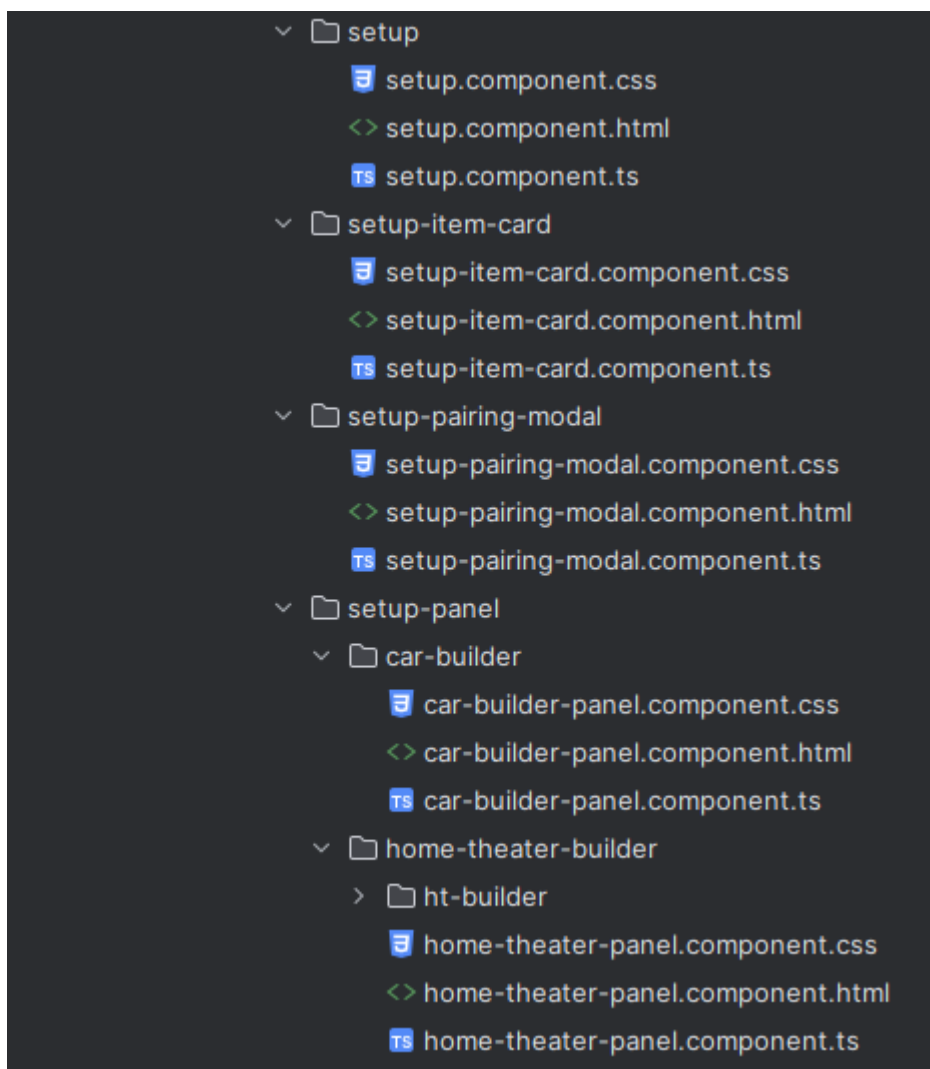


**33. Ábra: Tulajdonság sáv kinézete**

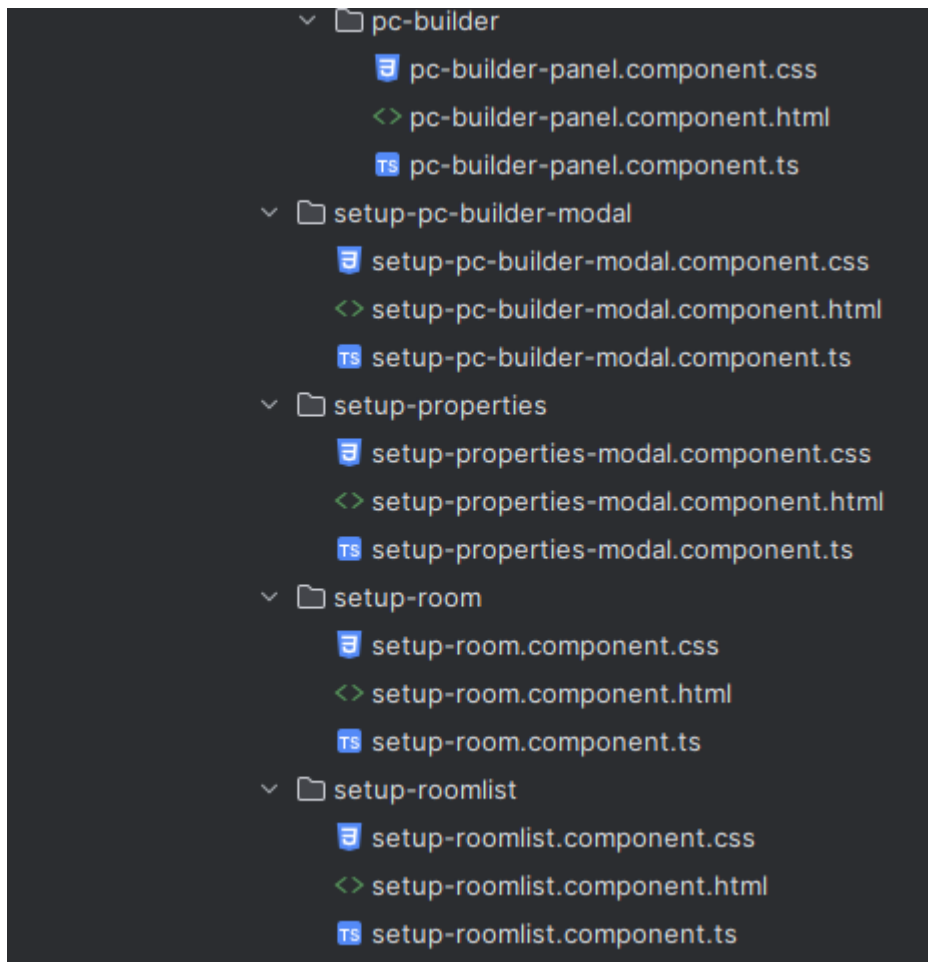
Mellékelt képek:



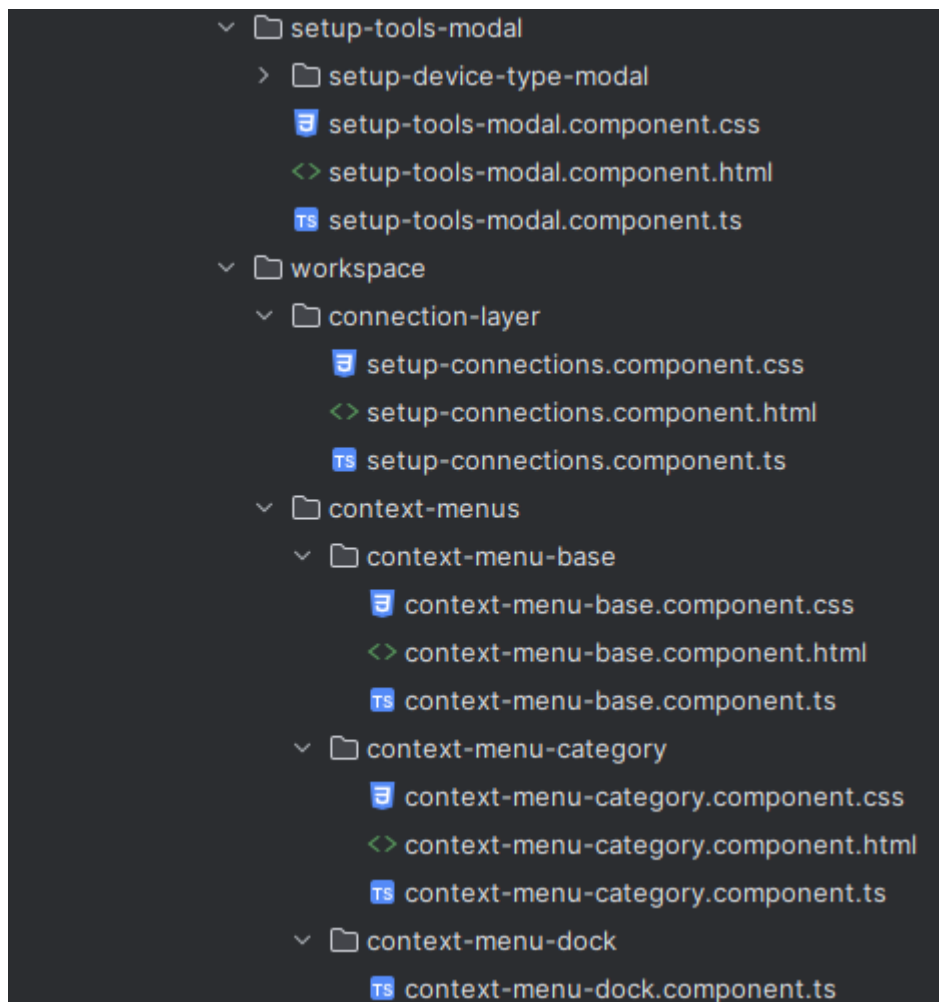
**34. Ábra: Az első kép a felépítésről**



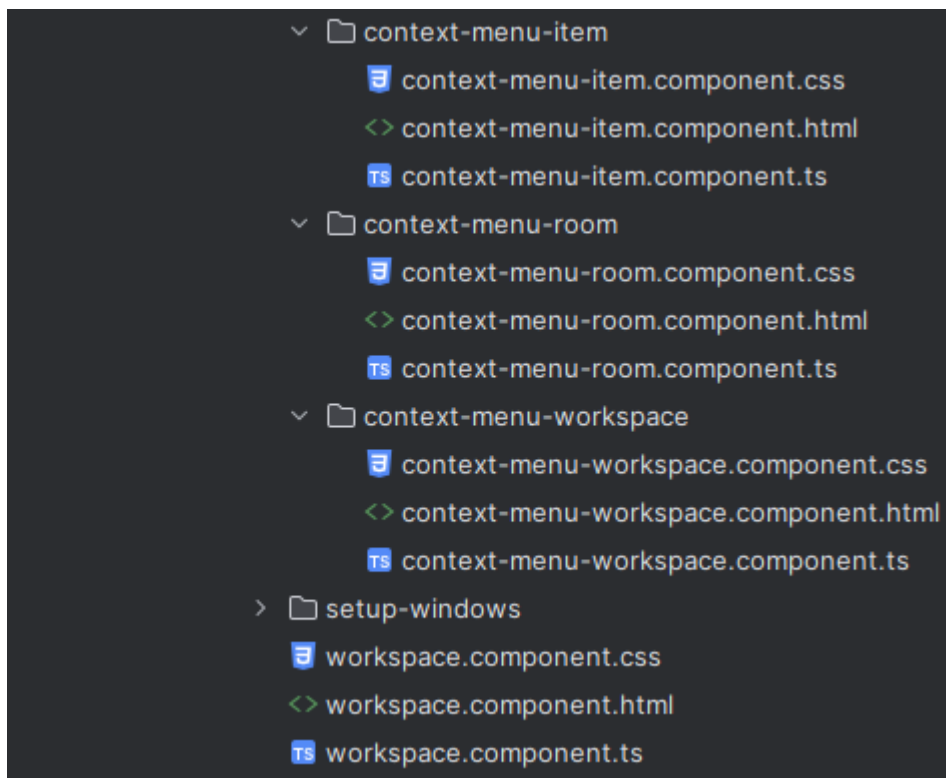
**35. Ábra: A második kép a felépítésről**



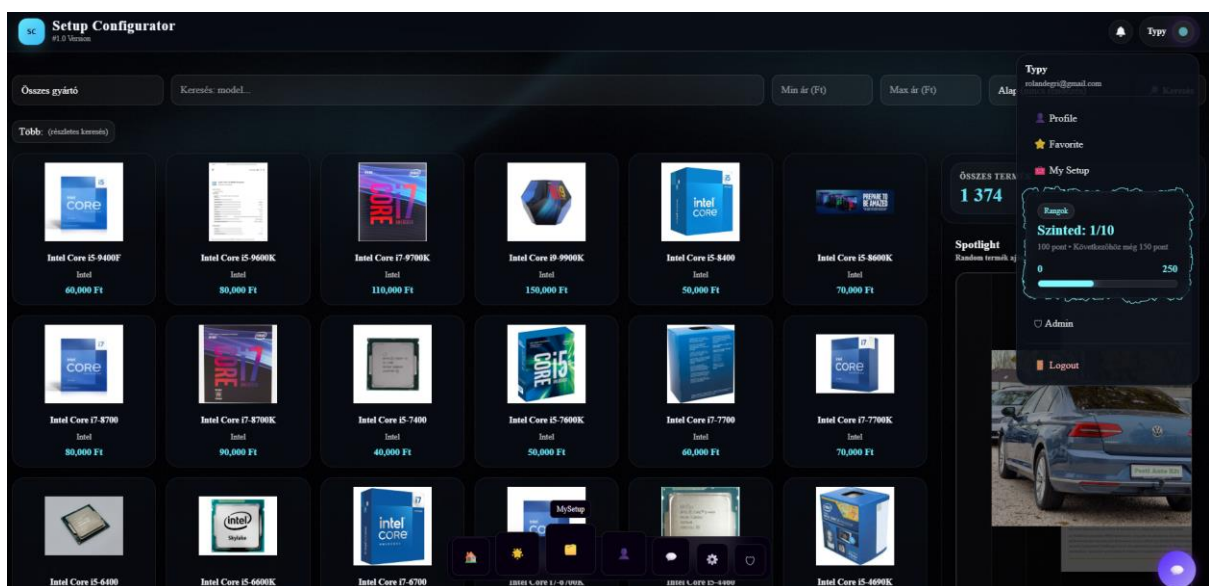
**36. Ábra: A harmadik kép a felépítésről**



**37. Ábra: A negyedik kép a felépítésről**



**38. Ábra: Az ötödik kép a felépítésről**



**39. Ábra: Profilos Mysetup**

## 7. Tesztelés

A fejlesztés során folyamatosan teszteltük az alkalmazás működését annak érdekében, hogy a különböző frontend funkciók megfelelően működjenek, valamint a felhasználói felület stabil és jól használható maradjon. A tesztelés célja az volt, hogy még a véglegesítés előtt kiszűrjük az esetleges hibákat, megjelenítési problémákat, logikai eltéréseket és működésbeli hiányosságokat.

A frontend tesztelése során külön figyelmet fordítottunk arra, hogy az egyes komponensek megfelelően kommunikáljanak egymással, és a felhasználói műveletek minden esetben a várt eredményt adják vissza. Ennek keretében ellenőriztük a főoldal terméklistájának megjelenítését, a keresési és szűrési rendszer működését, a lapozási logikát, a **Spotlight** ajánlót, az oldalsó termékadat-panel megnyitását, valamint a rangrendszer és ranglista működését is.

A tesztelés több szinten történt. Egyrészt minden új funkció fejlesztése után külön ellenőriztük annak helyes működését, másrészt a nagyobb rendszerelemek összeépítése után integrációs jellegű ellenőrzést is végeztünk. Ez azért volt fontos, mert egy frontend alkalmazásban nem elegendő önmagában csak egy-egy gomb vagy funkció tesztelése, hanem azt is vizsgálni kell, hogy az adott komponens hogyan viselkedik más elemekkel együtt.

A tesztelés során manuális ellenőrzést alkalmaztunk, vagyis minden funkciót több alkalommal végigpróbáltunk a felhasználói működés szempontjából. Ez különösen fontos volt az olyan interaktív elemeknél, mint a szűrők, a kategóriaváltás, a **Spotlight** blokk, a **dock** menü és az oldalsó termékpanel.

A rendszer működését különböző böngészőkben is ellenőriztük, például **Google Chrome** és **Microsoft Edge** alatt, hogy megbizonyosodjunk arról, hogy a felület és a funkcionalitás több környezetben is megfelelően működik. Emellett figyeltünk arra is, hogy a nagyobb mennyiségű adat kezelése – különösen a közel 1374 termék megjelenítése – ne okozzon teljesítménybeli problémát.

A tesztelési folyamat során az esetleges hibákat folyamatosan javítottuk. Több alkalommal előfordult például, hogy egyes termékek ára vagy képe nem megfelelően jelent meg, illetve bizonyos szűrési kombinációk esetén a lista nem a várt eredményt adta vissza. Ezeket a hibákat a kód módosításával, valamint az adatkezelés pontosításával javítottuk.

Összességében a tesztelési folyamat jelentősen hozzájárult ahhoz, hogy a frontend rész stabilabbá, gyorsabbá és felhasználóbarátabbá váljon. A fejlesztés végére a fő funkciók megbízhatóan működtek, és az oldal használata során nem jelentkeztek kritikus hibák.

## 7.1 Egységtesztek

Az alkalmazás fejlesztése során a különböző funkcionális egységeket külön is teszteltük. A tesztelés során ellenőriztük a termékek betöltését az adatbázisból, valamint a termékkártyák megfelelő megjelenítését.

Külön teszteltük a keresési és szűrési funkciókat, amelyek lehetővé teszik a felhasználók számára, hogy gyártó, ár vagy kategória alapján találjanak termékeket.

Összes gyártó

Keresés: model...

Több: (részletes keresés)

Autók

Computer

HT

Instr

Autók

Csak autó táblákra szűr

BMW

Modell (pl. 320d)

1000000

Ár max (Ft)

Kivitel (összes)

Lóerő min

400

0–100 min (s)

0–100 max (s)

Ülések min

Ülések max

Üzemanyag (összes)

Évjárat min

Évjárat max

Automata

Autó szűrők törlése

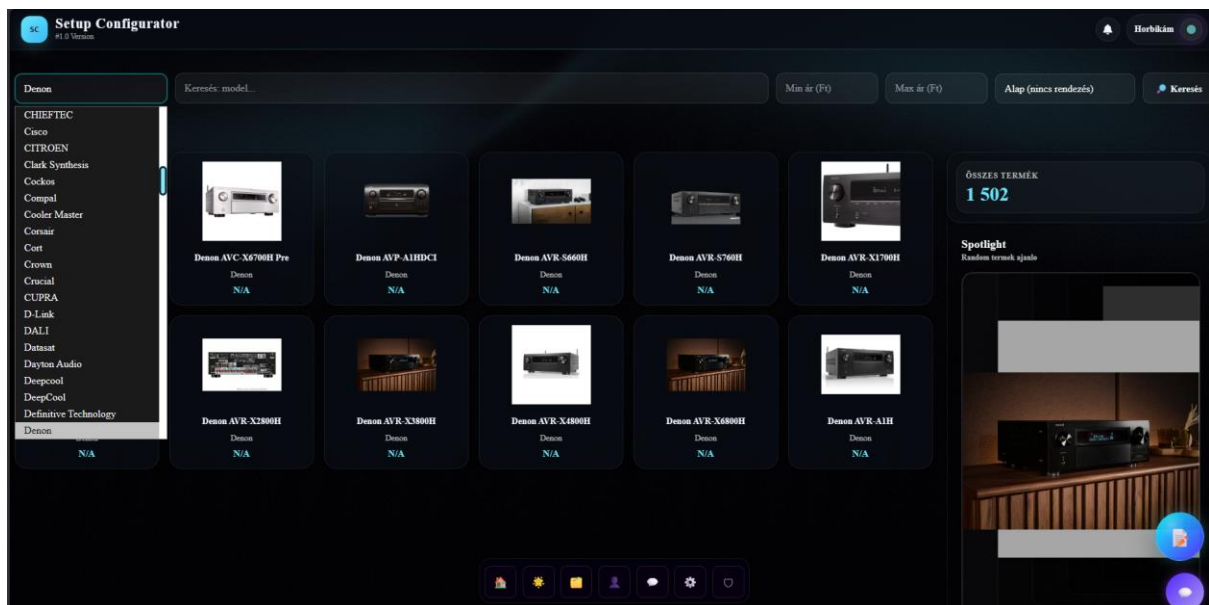
A Keresés gomb ezeket is figyelembe veszi.



BMW M4 Competition DKG  
BMW  
18,000,000 Ft



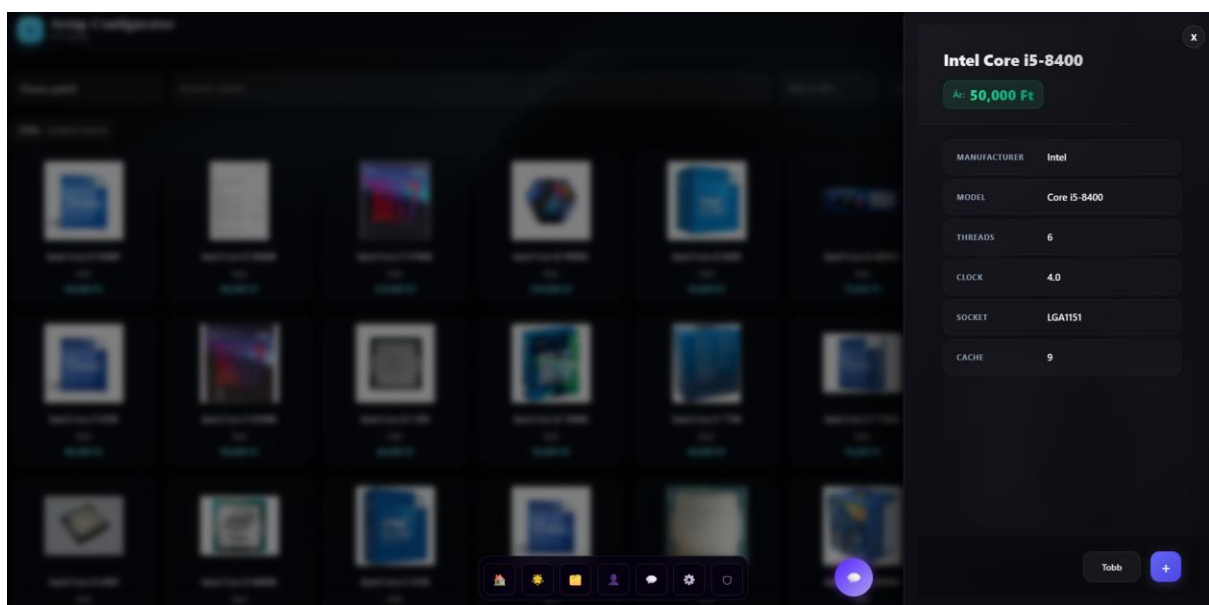
BMW 225e xDrive DKG  
BMW  
14,210,000 Ft



## 40. Ábra: Modell szűrő

*A fenti képen látható a külön modellekre létrehozott szűrő, ami kiválóan működőképes.*

Tesztelésre került továbbá a termék részleteit megjelenítő oldalsó panel működése, a lapozási rendszer, amely egyszerre 50 terméket jelenít meg, valamint a Spotlight termékajánló funkció.



ÖSSZES TERMÉK

1 502

### Spotlight

Random termék ajánló



LTD

**LTD M-200DX Purple Burst**

**195 548 Ft**

table: products

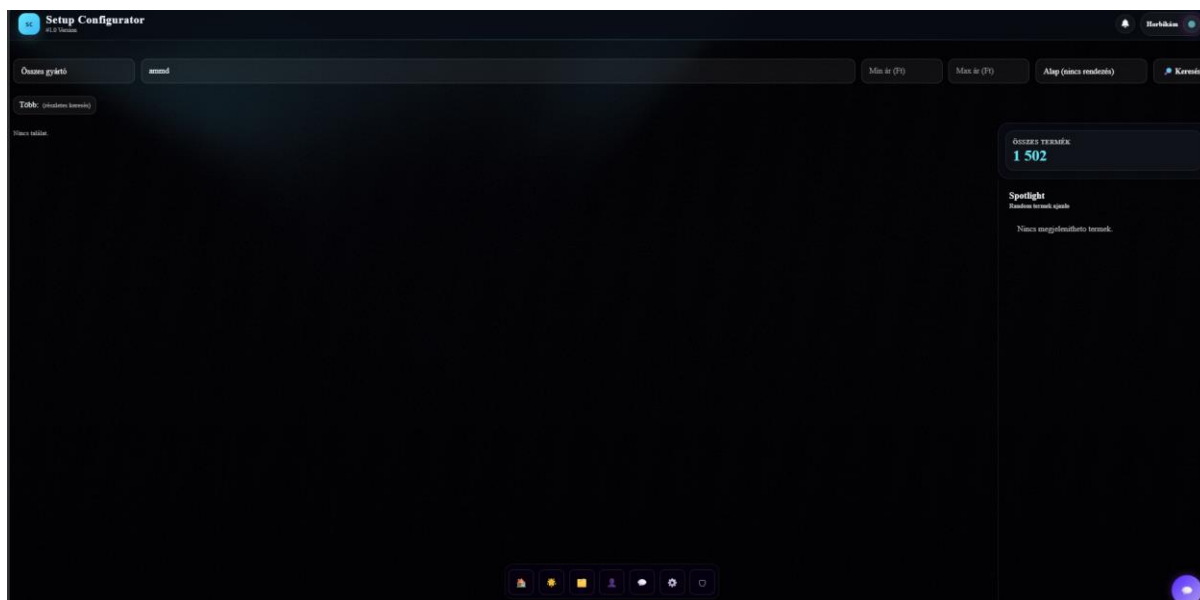
id: 1054

|              |                      |
|--------------|----------------------|
| manufacturer | LTD                  |
| model        | M-200DX Purple Burst |
| guitar_type  | electric             |
| color        | Purple Burst         |
| string_count | 6                    |

kattints a kártyára a részletekhez

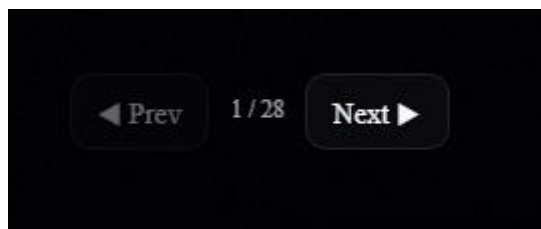


A tesztelés során ellenőriztük azt is, hogy a rendszer hogyan reagál hibás vagy hiányos adatok esetén. Például teszteltük a keresőmezőt üres kereséssel, illetve olyan kifejezésekkel, amelyekhez nem tartozik termék az adatbázisban. A rendszer ilyen esetekben megfelelően kezelte a helyzetet, és nem okozott hibát a működésben.



**41. Ábra: Hibás adatú kereső**

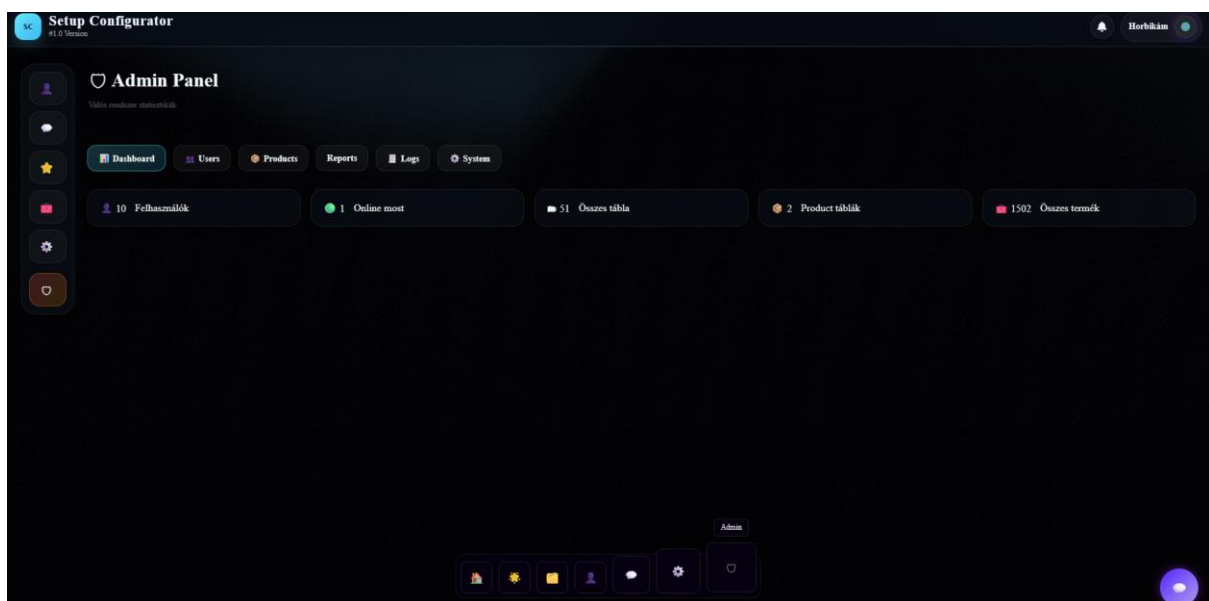
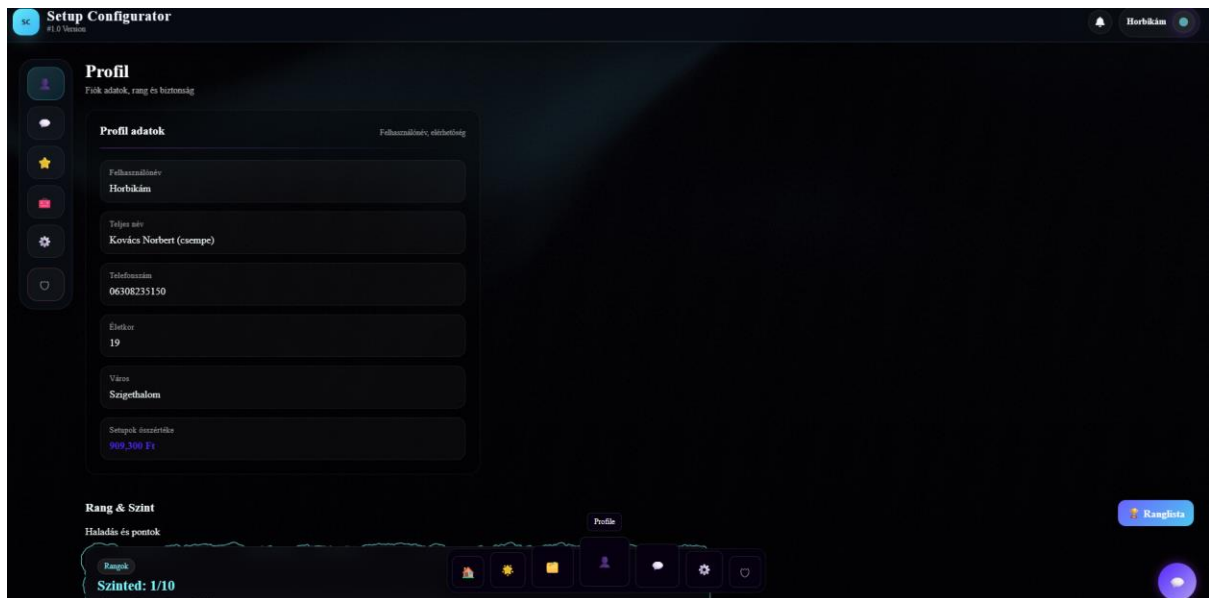
A rendszer teljesítményét is ellenőriztük nagyobb mennyiségű adat esetén. Az adatbázis jelenleg közel 1502 terméket tartalmaz, ezért fontos volt, hogy az oldal ne próbálja meg egyszerre betölteni az összes elemet. Ennek érdekében lapozási rendszert alkalmaztunk, amely egyszerre 50 terméket jelenít meg. A tesztelés során az oldal stabilan működött, és nem tapasztaltunk jelentős lassulást.



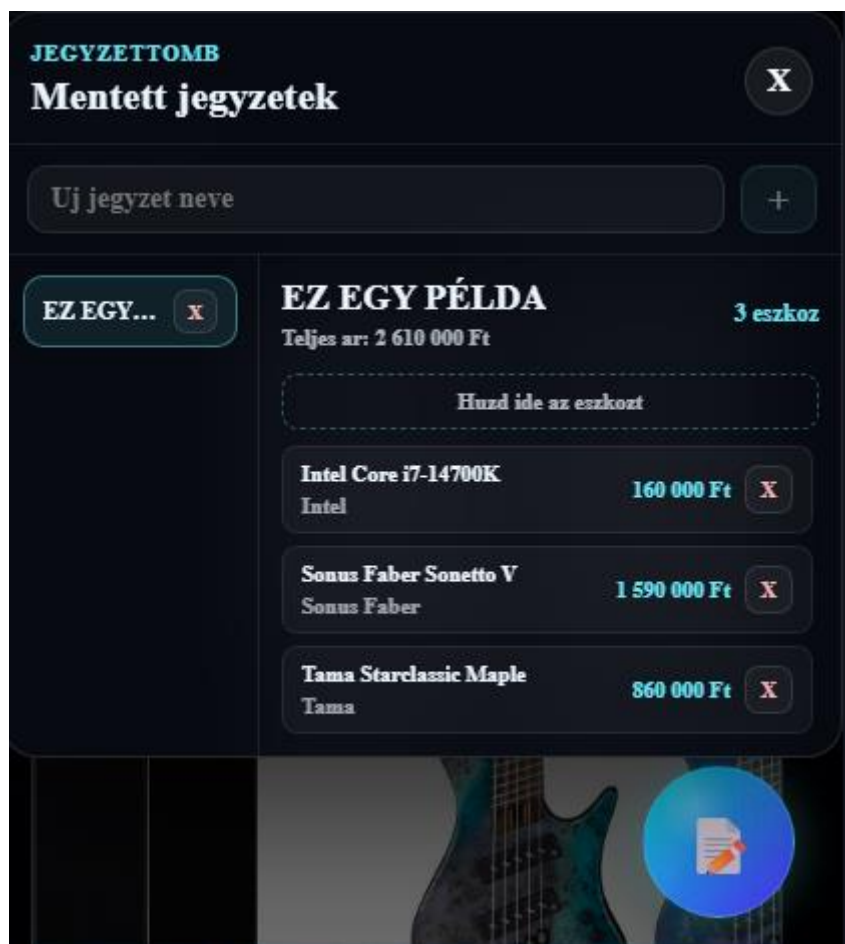
**42. Ábra: Oldal léptető**

A fejlesztés végén felhasználói tesztelést is végeztünk. Az alkalmazást több különböző eszközön és böngészőben is kipróbáltuk, például **Google Chrome** és **Microsoft Edge** böngészőkben. A tesztelés

során a felhasználók képesek voltak könnyen navigálni az oldalon, valamint használni a keresési és szűrési funkciókat.



***Továbbá az alul látható Dock is megfelelően működik, ha bármelyik gyorsshivatkozásra rányom az illető, az összes oldal sikeresen betöltődik.***



### 43. Ábra: Jegyzettömb

*A fenti képen látható a jegyzettömb, ahova behúzhatja, és elmentheti a felhasználó azokat a termékeket, amik érdeklik, ez a funkció is működik.*

Írd le az Asrock B650 Steel Legend adatait

AI Valasz

17:48

Íme az ASRock B650 Steel Legend alaplap részletes adatai a katalógusunkból:

Ez egy modern, felső-középkategóriás alaplap, amely az AMD legújabb platformjára épül.

- Ár: 110 000 Ft
- Foglalat (Socket): AM5 (AMD Ryzen 7000-es és 8000-es szériához)
- Chipset: B650
- Méret szabvány: ATX
- Memória: DDR5 típusú, 4 foglalattal rendelkezik, és összesen maximum 192 GB RAM-ot kezel.
- Adattárolás és bővítés:
- PCIe verzió: PCIe 5.0 (a leggyorsabb jelenlegi szabvány)
- M.2 foglalatok száma: 3 db (gyors NVMe SSD-knek)
- SATA portok: 4 db
- Hálózat és kapcsolat:
- LAN sebesség: 2.5G (szupergyors vezetékes net)
- Vezeték nélküli: Beépített WiFi és Bluetooth
- USB portok: 10 db
- Egyéb: RGB világítás támogatás.

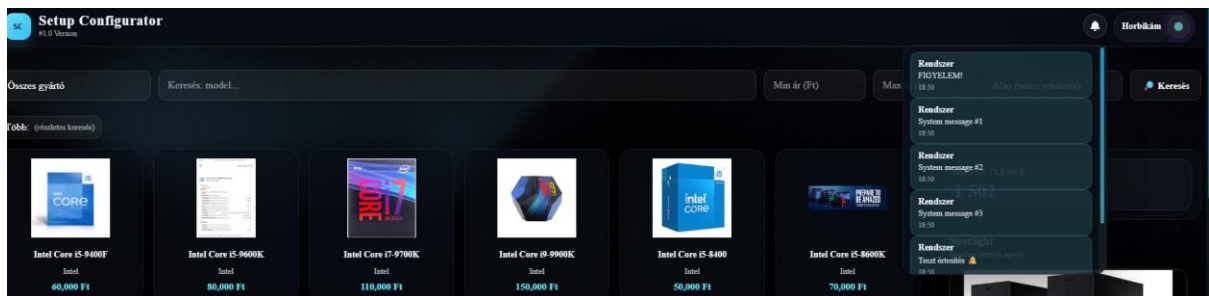
Ez az alaplap kiváló választás, ha egy jövőálló gépet szeretnél építeni, mivel támogatja a legújabb DDR5 memóriákat és a PCIe 5.0-s eszközöket is.

Írj kerdest a termékekről...

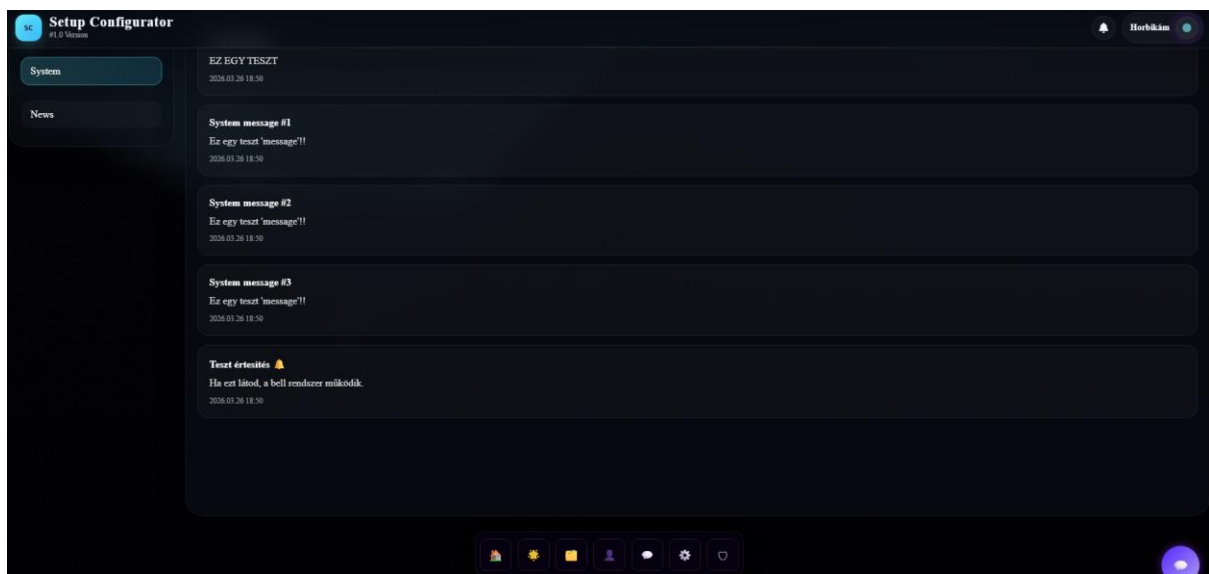
Kuldes

## 44. Ábra: AI

A fenti képen látható a beépített AI egységtesztje, ahol most például kiírtattuk az egyik alaplap tulajdonságait, ahogy látszik a képen, ez sikeresen megtörtént.

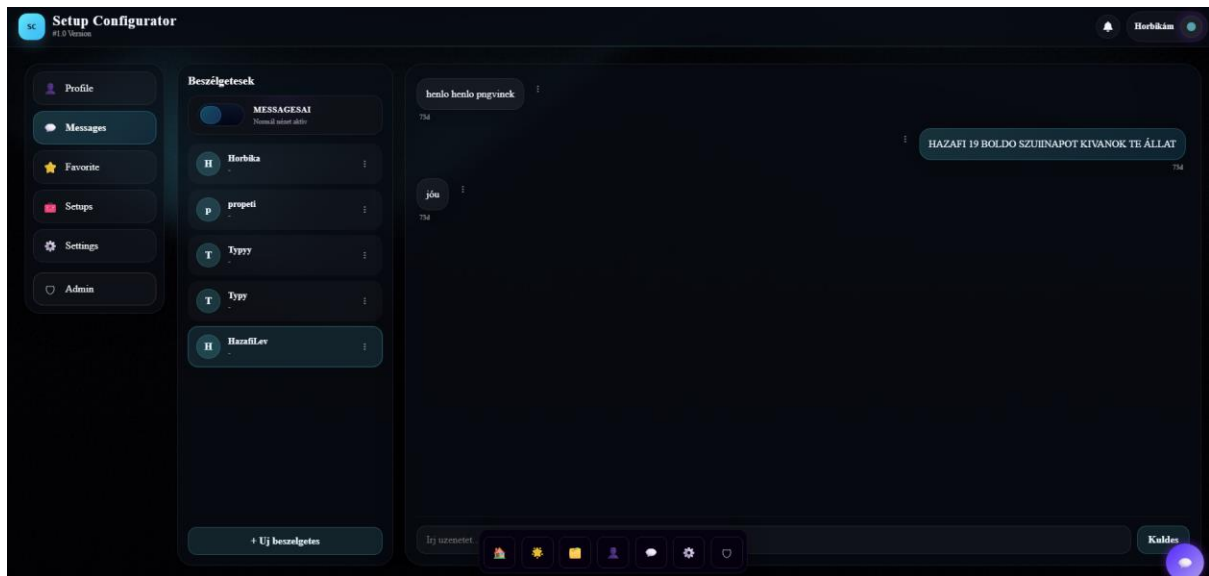


Itt látható, a Bell (értesítés) amire rányomva megnyílik egy másik oldalon, ha az admin kiküld egy frissítést, automatikusan megjelenik a felhasználónak is.



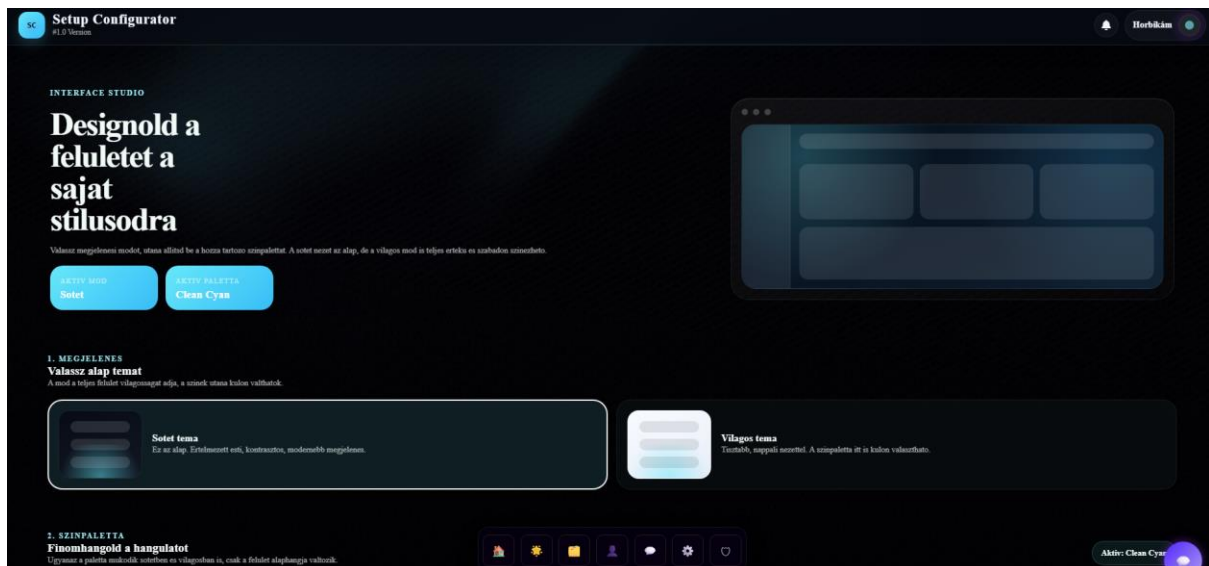
## 45. Ábra: Értesítés

A lenti képen látható, az "üzengető fal", ahol a felhasználók egymás között kommunikálhatnak szabadon, az elküldött üzenetet lehet szerkeszteni, törölni. Az üzenet biztonságosan, gyorsan megérkezik.

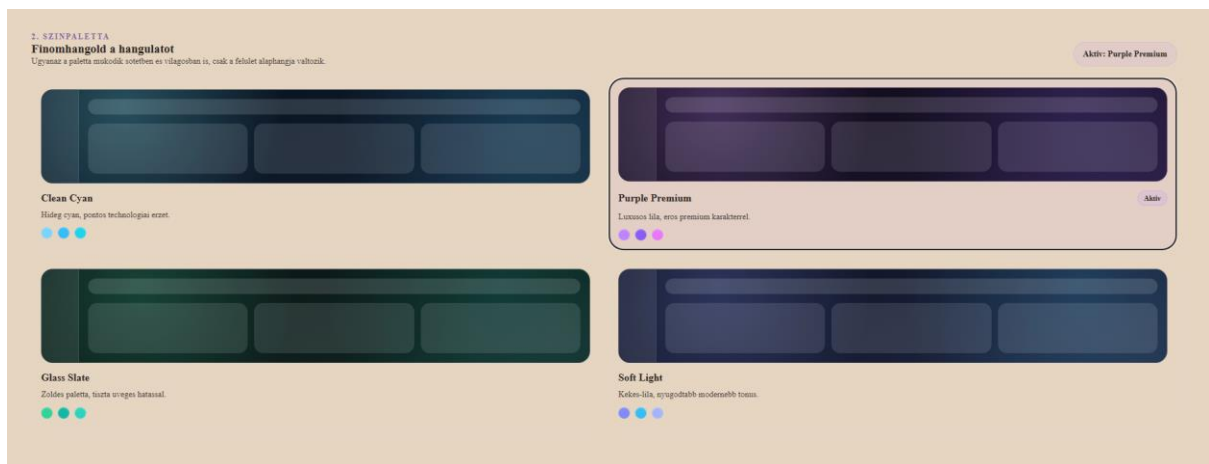
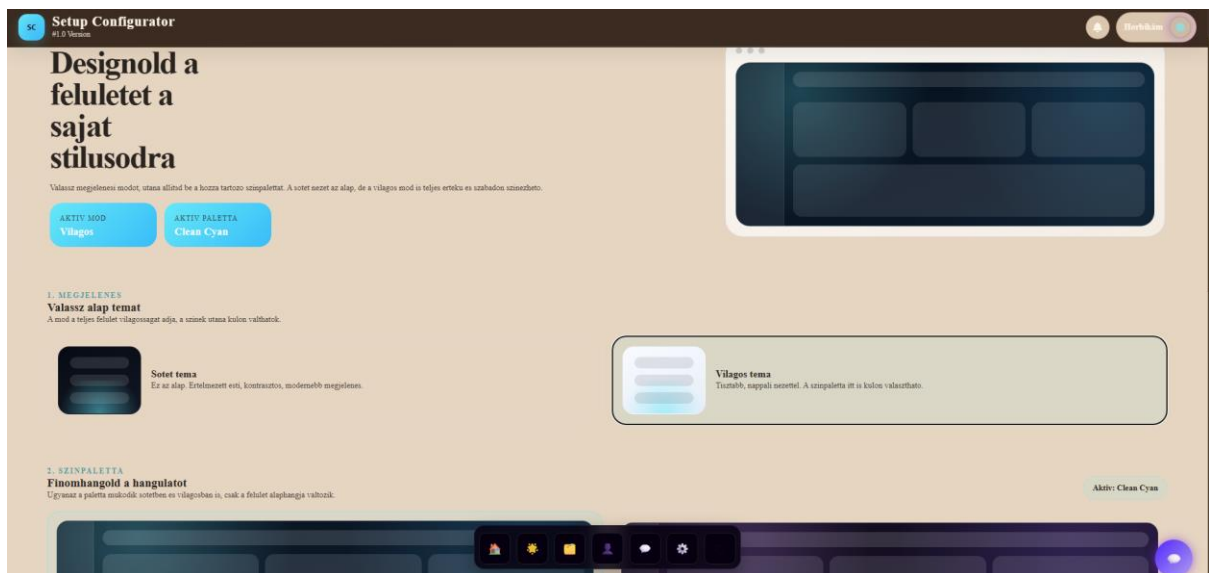


46. Ábra: Messages

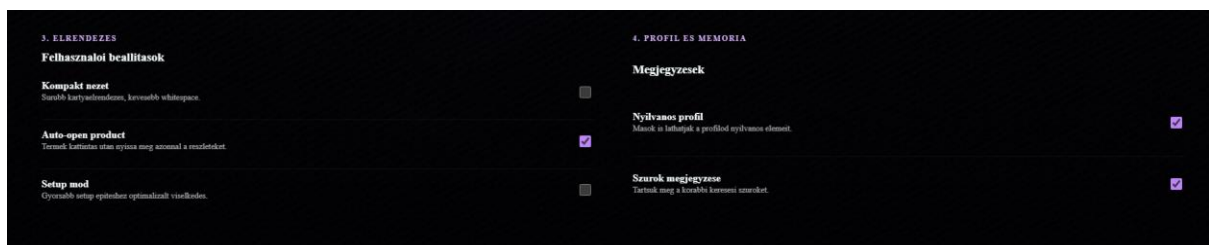
A lenti képen látható az oldalnak a settings (beállítás) része, ahol be lehet állítani az oldalnak bizonyos témáit. Ha az illető rányom valamelyik alap témára, akkor az oldalon ez megváltozik.



47. Ábra: Beállítások

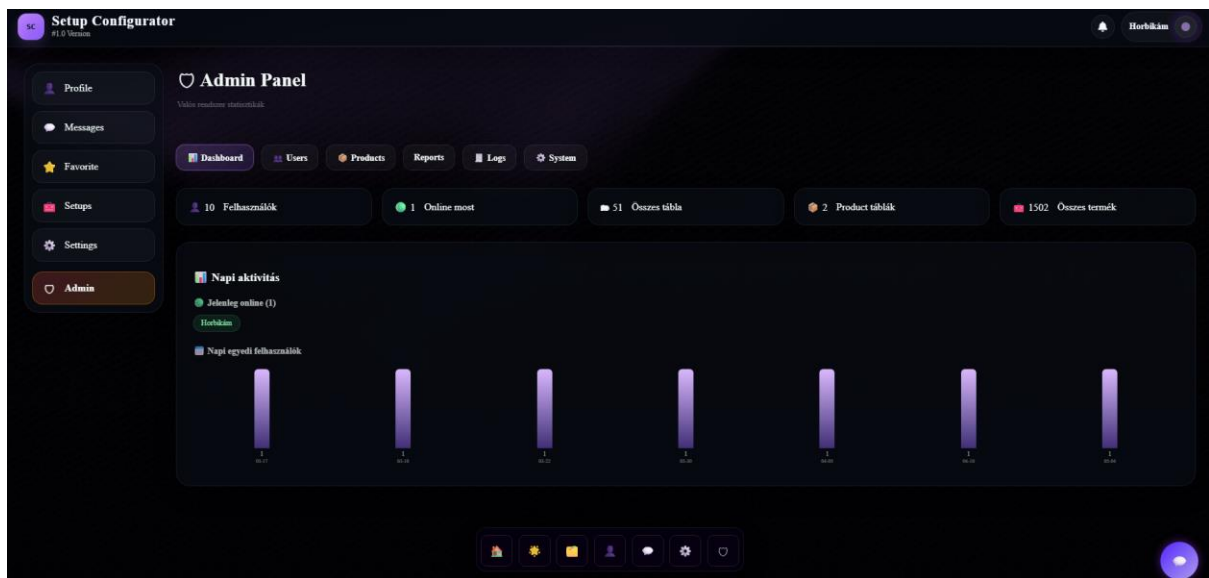


**Illetve vannak még részletesebb további beállítási funkciók, ezek például az oldalknak, kis részleteinek a színéért felelnek.**



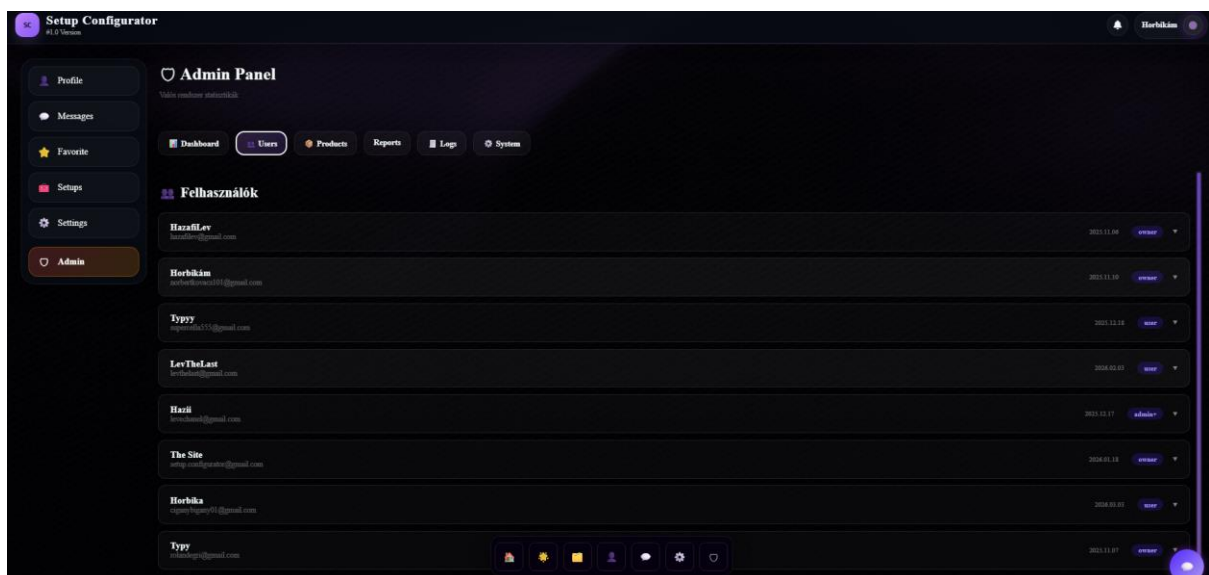
## 48. Ábra: További Beállítások

**Utoljára itt vannak funkciók, amik által el lehet érni azt, hogy elmentse a szűrőket, privát, vagy publikus legyen a profil más számára, kompaktabb nézet.**



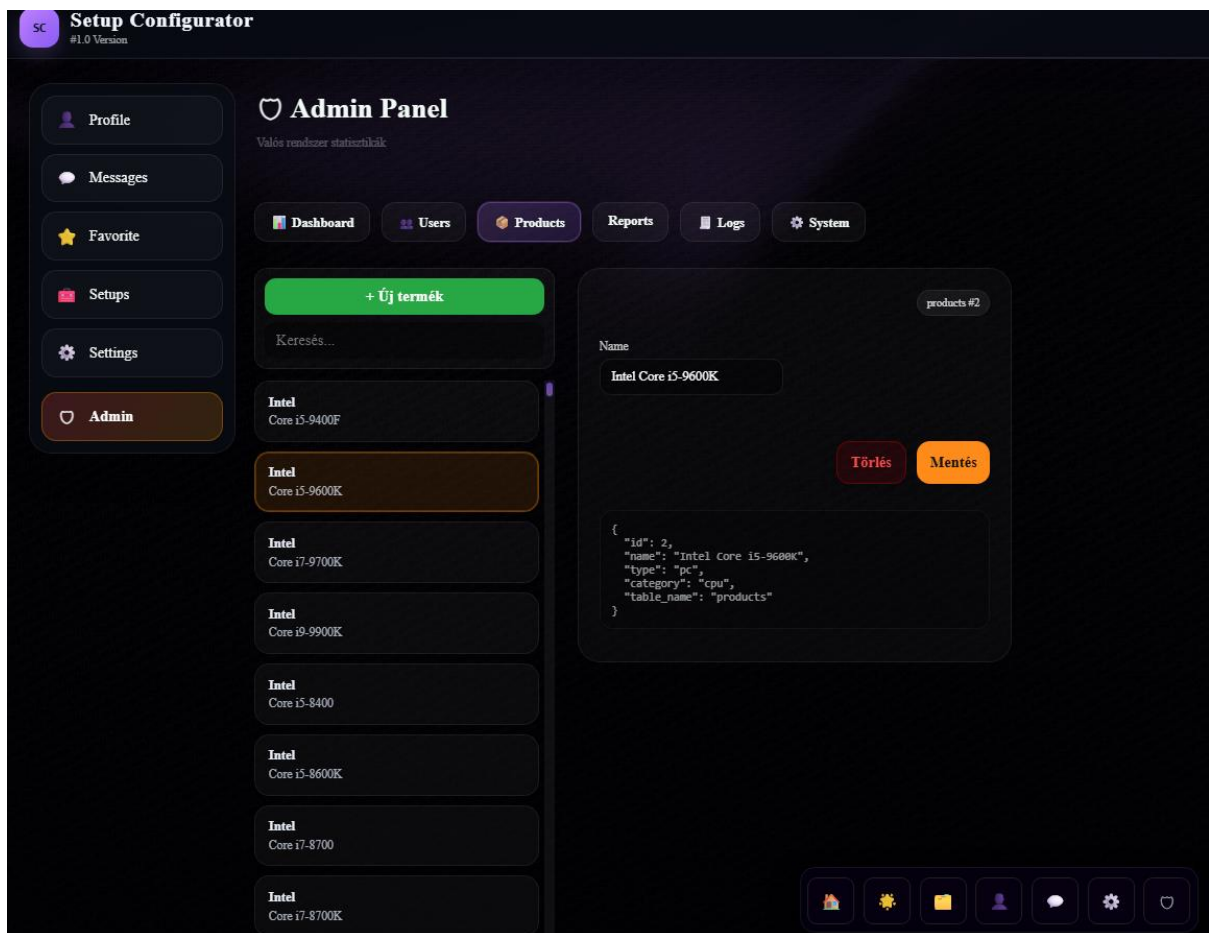
**49. Ábra: Admin oldal**

*Itt található az Admin fül, ahol a felhasználó (ha van/kap olyan rangot), akkor láthatja az oldal részleteit/tulajdonságait. Látható például hogy kik/hányan vannak Online az oldalon jelenleg, hogy az adatbázisban (Supabasen) hány tábla van összesen, ebből hány tábla product, illetve hogy hány termék van összesen.*



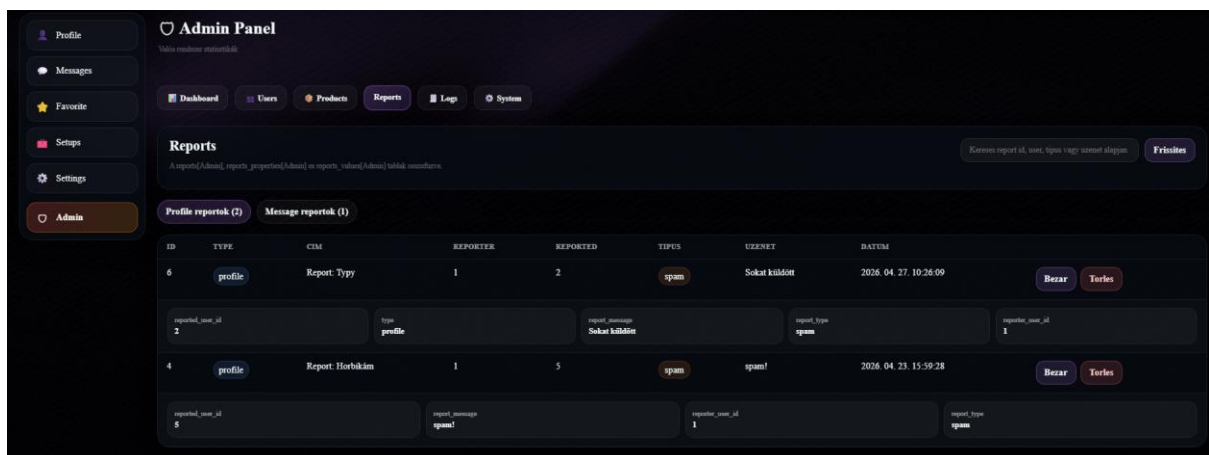
**50. Ábra: Admin - User panel**

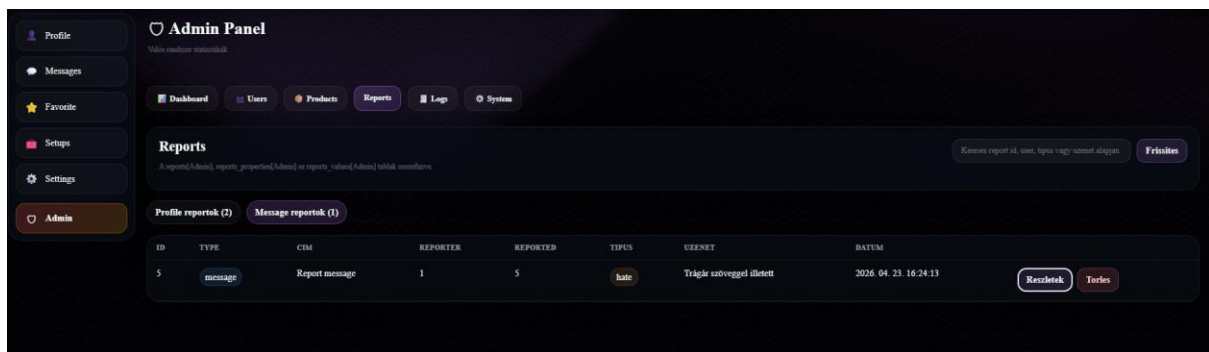
*Ez egy külön User Panel, ahol lehet látni az összes felhasználót, aki valaha regisztrált az oldalra, és az elérhetőségét (email).*



**51. Ábra: Admin - Product panel**

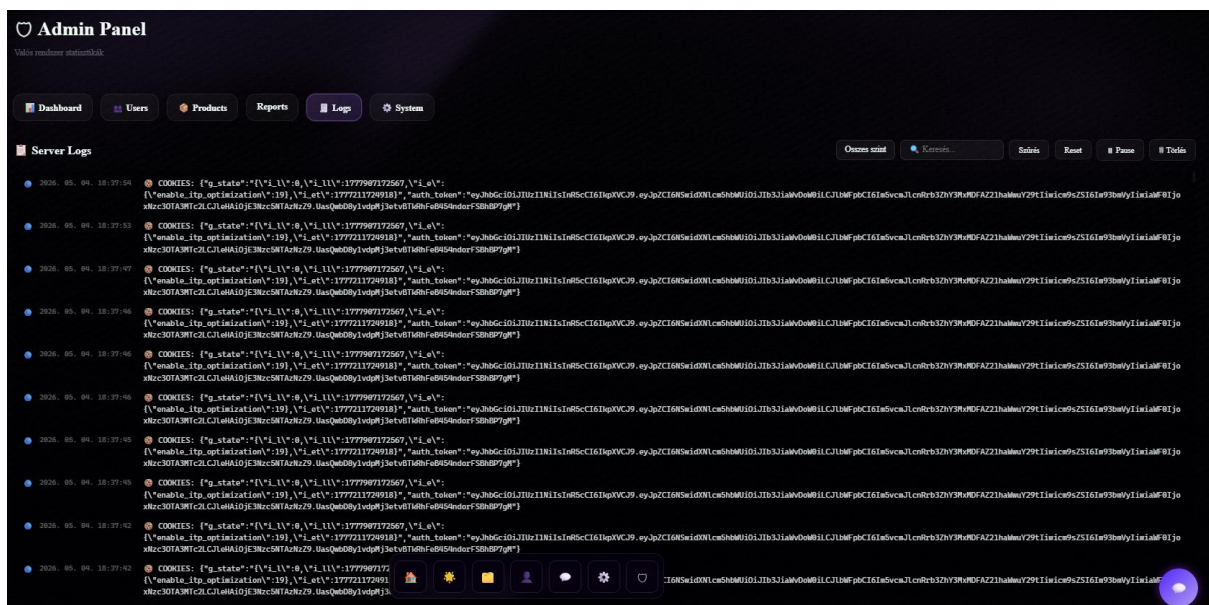
**A Product Panelben lehet módosítani, törölni, hozzáadni egy terméket, ami az adatbázisban is megjelenik.**





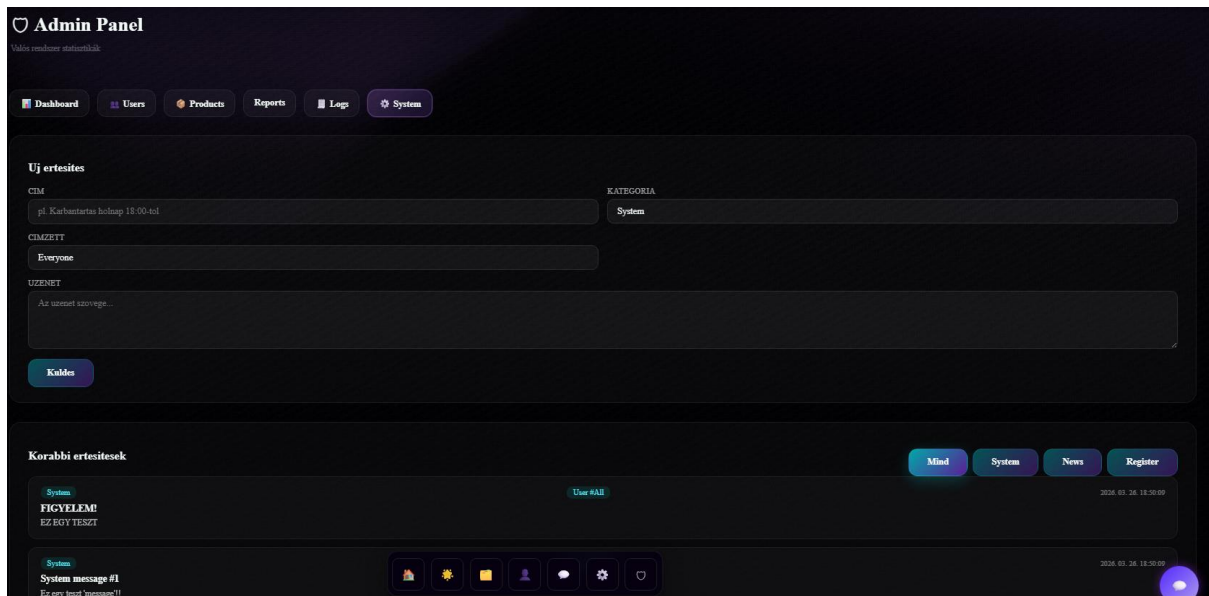
**52. Ábra: Admin - Reports panel**

Ezekon a képeken látható a Report (Bejelentés) Panel, ahol akár Profil, és Message bejelentések jelennek meg bizonyos dolgok miatt. Pl. Spam, túl sok üzenet küldése, Trágár beszéd.



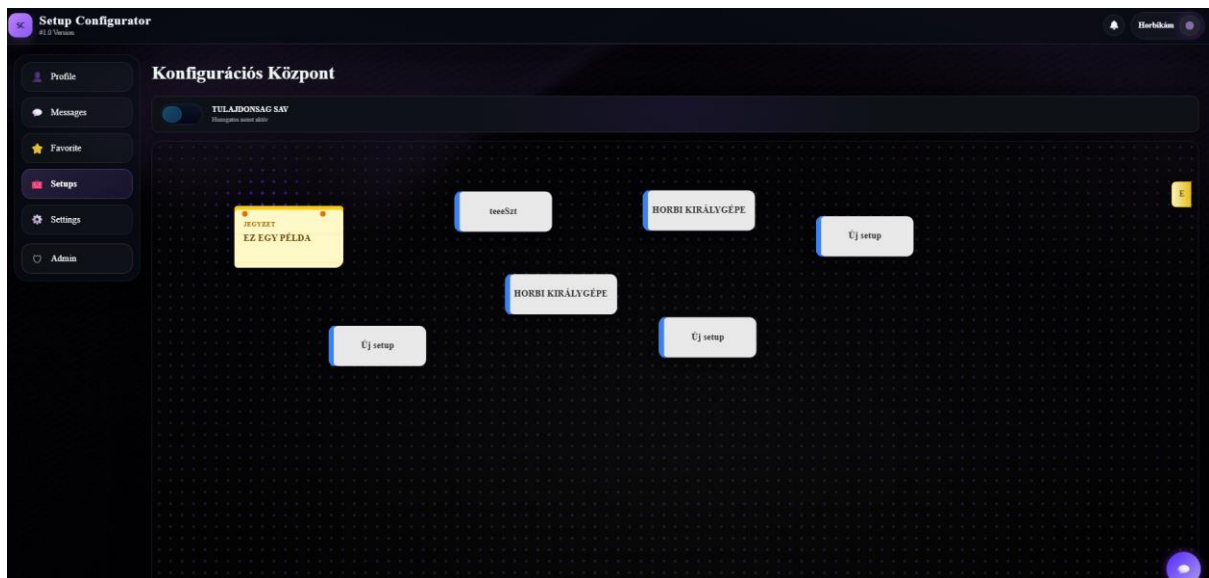
**53. Ábra: Admin - Log panel**

Itt látható az oldal összes logja kiírva amit lehet szűrni logok, figyelmeztetés, hiba, vagy manuális keresés alapján. Ezeket lehet törölni, visszaállítani, vagy megállítani.



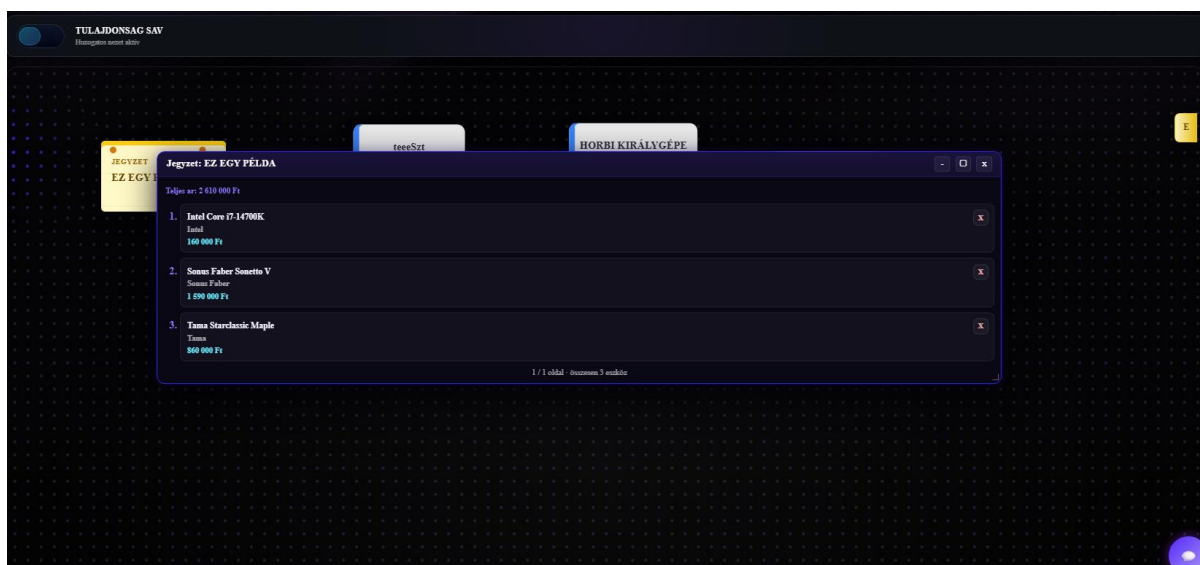
**54. Ábra: Admin - System panel**

**Végül van a System Panel, ahol lehet kiküldeni a felhasználóknak üzenetet, értesítéseket, persze külön a címzett fülnél lehet állítani azt, hogy kik kapják meg az értesítést.**



**55. Ábra: Admin - Setup panel**

*Ezen a képen látható a Setups, vagyis a Konfigurációs Központ, ahol lehet jobb klikkel új setupot létrehozni, összeállítani. A jegyzetömb, amit az elején láthattunk az itt is megjelenik, egy cetli alapú kinézettel, hogy meg lehessen különböztetni. Ha bal klikkel rákattintunk a jegyzetömbre kétszer, akkor megnyílik külön egy kis ablak, amit le lehet tálcázni, be lehet zárni, illetve lehet rendezni különböző fajta módon. A letálcázott jegyzetömböket a jobb oldalt látható sárga színű “könyvelemszerű” gombbal lehet előhozni.*



## 7.2 Teszteredmények dokumentálása

A tesztelési folyamat során a rendszer fő funkciói megfelelően működtek. A termékek betöltése az adatbázisból stabilan működött, valamint a keresési és szűrési funkciók lehetővé tették a felhasználók számára, hogy különböző szempontok alapján találjanak termékeket. A termék részleteit megjelenítő oldalsó panel és a lapozási rendszer szintén megfelelően működött.

A tesztelés során néhány kisebb hiányosság is megfigyelhető volt. Egyes termékek esetében az ár adat nem állt rendelkezésre az adatbázisban, ezért ezek az elemek ár nélkül jelennek meg. Emellett bizonyos gyártók neve több alkalommal is szerepelhet a keresési találatok között.

Néhány termékhez tartozó kép jelenleg nem jelenik meg, mivel az adott képek nem állnak szabadon felhasználható formában rendelkezésre, vagy szerzői jogi korlátozás alá esnek.

Ezek a hiányosságok azonban nem befolyásolják jelentősen a rendszer alapvető működését, és a fő funkciók a tesztelés során stabilan működtek.

| <b><i>Tesztelt funkció</i></b> | <b><i>Teszt leírása</i></b>                             | <b><i>Várt Eredmény</i></b>                              | <b><i>Eredmény</i></b>                                                                                  |
|--------------------------------|---------------------------------------------------------|----------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| Terméklista betöltése          | Oldal megnyitása után a termékek betöltése adatbázisból | A termékkártyák megjelennek                              | Megfelelő, a termékek betöltődnek az adatbázisból                                                       |
| Keresés                        | Gyártó név beírása a kereső mezőbe                      | A rendszer csak a megfelelő termékeket jeleníti meg      | Részből megfelelő, néhány irreleváns találat is megjelenik.                                             |
| Szűrés                         | Ár szerinti szűrés használata                           | Csak a megadott ár tartományba eső termékek jelennek meg | Részből megfelelő, az ár szerinti szűrés működik, de hiányos adatoknál pontatlan lehet                  |
| Termék részletek Panel         | Termékkártyára kattintás                                | Megjelenik az oldalsó panel a termék adataival           | Megfelelő, az oldalsó panel megjeleníti a termék adatait.                                               |
| Lapozás                        | Következő oldal gomb használata                         | A következő 50 termék jelenik meg                        | Megfelelő, a következő oldal termékei betöltődnek.                                                      |
| Spotlight                      | Ajánló rész frissítése                                  | Véletlenszerű termék jelenik meg animációval             | Részből megfelelő, az ajánlott termékek megjelennek, de a véletlenszerűség miatt eltérő eredményt adhat |
| Setup                          | Setup létrehozása                                       | Létrehozunk egy Setupot.                                 | A setup sikeresen létre lett hozva.                                                                     |
| Setup                          | Setup megnyitása                                        | Megnyitjuk a setupot.                                    | A setup sikeresen megnyílt.                                                                             |
| Setup                          | Setup jobb klikk esemény funkció                        | Rányomunk a setupra jobb klikkel.                        | Jobb klikk kattintásakor a setupra megnyitott                                                           |

|                 |                                          |                                                                            |                                                                        |
|-----------------|------------------------------------------|----------------------------------------------------------------------------|------------------------------------------------------------------------|
|                 |                                          |                                                                            | egy legördülő ablakot, különböző funkciókkal.                          |
| Setup           | Setup eszközök megjelenítése             | Setupon belül megjelennek-e az adott eszköz/eszközök.                      | A setup megnyitásakor, a belerakott eszközök látszódnak.               |
| Setup           | Setup átnevezése                         | Egy setup átnevezése másra.                                                | A setup sikeresen átneveződött probléma nélkül.                        |
| Setup           | Setup összekötések                       | Setupon belüli eszközök összekötése.                                       | A setupon belüli eszközök sikeresen összekötődik egymáshoz.            |
| Setup           | Setup kapcsolatok                        | Setupon belüli kapcsolatok megjelenítése.                                  | A setupon belül sikeresen megjelentek az eszközök közötti kapcsolatok. |
| Setup           | Setup törlés                             | Egy setup törlése.                                                         | A setup sikeresen törölődött, miután rányomunk a törlés gombra.        |
| Setup           | Setup kártya mozgatása                   | Egy setup kártya szabadon mozgatása.                                       | A setup kártya egy bizonyos területen belül sikeresen mozgatható.      |
| Tulajdonság sáv | Tulajdonság sáv működése, funkcionálása. | Tulajdonság sáv működése, funkcionálása.                                   | Részben működőképes.                                                   |
| eszköz          | eszköz Létrehozása                       | Setupon belüli eszköz létrehozása                                          | Az eszköz sikeresen létre lett hozva a setupon belül.                  |
| eszköz          | eszköz jobb klikk esemény.               | Az eszközök jobb klikkre való nyomásra funkciók jelennek meg, és működnek. | Jobb klikkre sikeresen megnyílt, és működik az összes funkció.         |

|        |                   |                       |                                                |
|--------|-------------------|-----------------------|------------------------------------------------|
| eszköz | eszköz megnyitása | Az eszköz megnyitása. | Az eszköz probléma nélkül, sikeresen megnyílt. |
| eszköz | eszköz átnevezése | Az eszköz átnevezése  | Az eszköz sikeresen átneveződött.              |
| eszköz | eszköz módosítása | Az eszköz módosítása  | Az eszköz sikeresen módosult.                  |
| eszköz | eszköz törlése    | Az eszköz törlése     | Az eszköz sikeresen törlődött.                 |

## 8. Telepítési útmutató

Töltse le vagy klónozza a repository-t a gépére.

FONTOS a repository-t a main ágból kell letölteni, a többi ág nem releváns.

Nyissa meg a projekt mappáját, majd indítsa el a start.bat fájlt.

Fontos: a folyamat közben megnyíló parancssori (CMD) ablakokat ne zárja be, mert ezek futtatják a szervereket.

Első indításkor a script automatikusan:

ellenőrzi a Node.js + npm meglétét,

```

start.bat
18 call :install_deps "%ROOT%" "%NPM_CMD%"
19 if errorlevel 1 exit /b 1
20
21 call :install_deps "%ROOT%" Setup-Configurator "%NPM_CMD%"
22 if errorlevel 1 exit /b 1
23
24 call :verify_native_deps "%ROOT%" "%NODE_EXE%" "%NPM_CMD%"
25 if errorlevel 1 exit /b 1
26
27 call :free_backend_port "%ROOT%" "3000"
28 if errorlevel 1 exit /b 1
29
30 echo =====
31 echo Starting Backend Server...
32 echo =====
33
34 start "Backend Server" cmd /w call "%ROOT%\tools\run-with-repair.cmd" "%ROOT%" "%NODE_EXE%" "%NPM_CMD%" "%ROOT%\backend" "%NODE_EXE%" server.js
35
36 echo =====
37 echo Starting Angular Server...
38 echo =====
39
40 start "Angular Server" cmd /w call "%ROOT%\tools\run-with-repair.cmd" "%ROOT%" "%NODE_EXE%" "%NPM_CMD%" "%ROOT%\Setup-Configurator" "%NPM_CMD%"
41
42 echo =====
43 echo Starting Admin Console...
44 echo =====

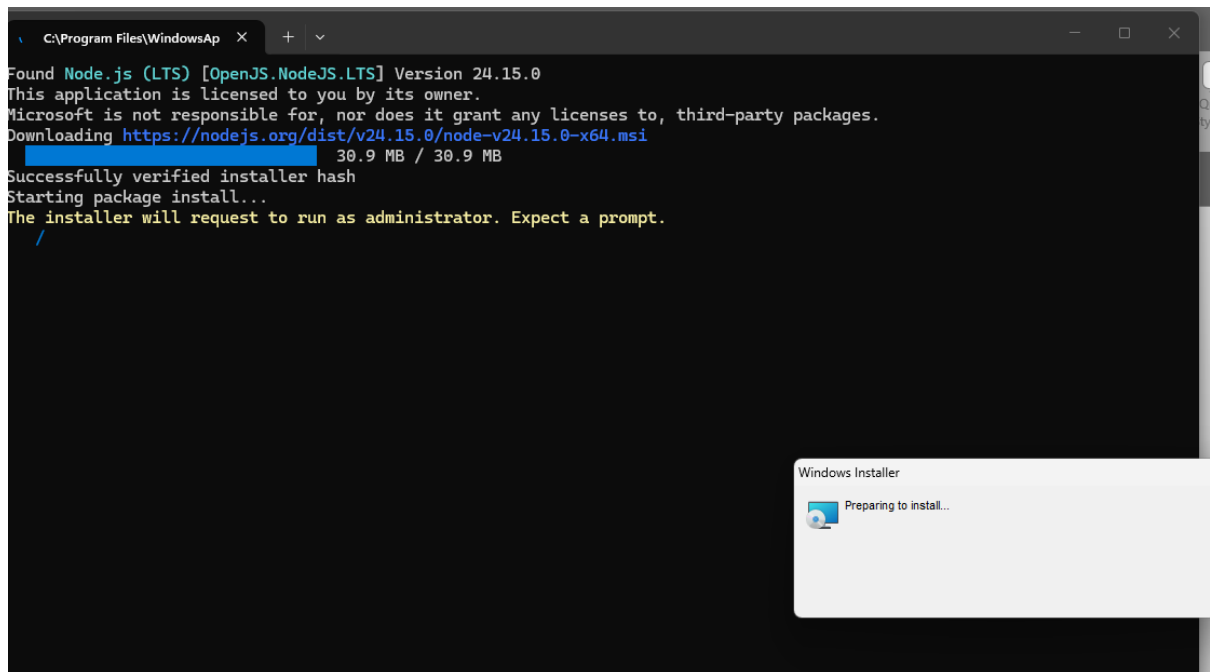
```

```

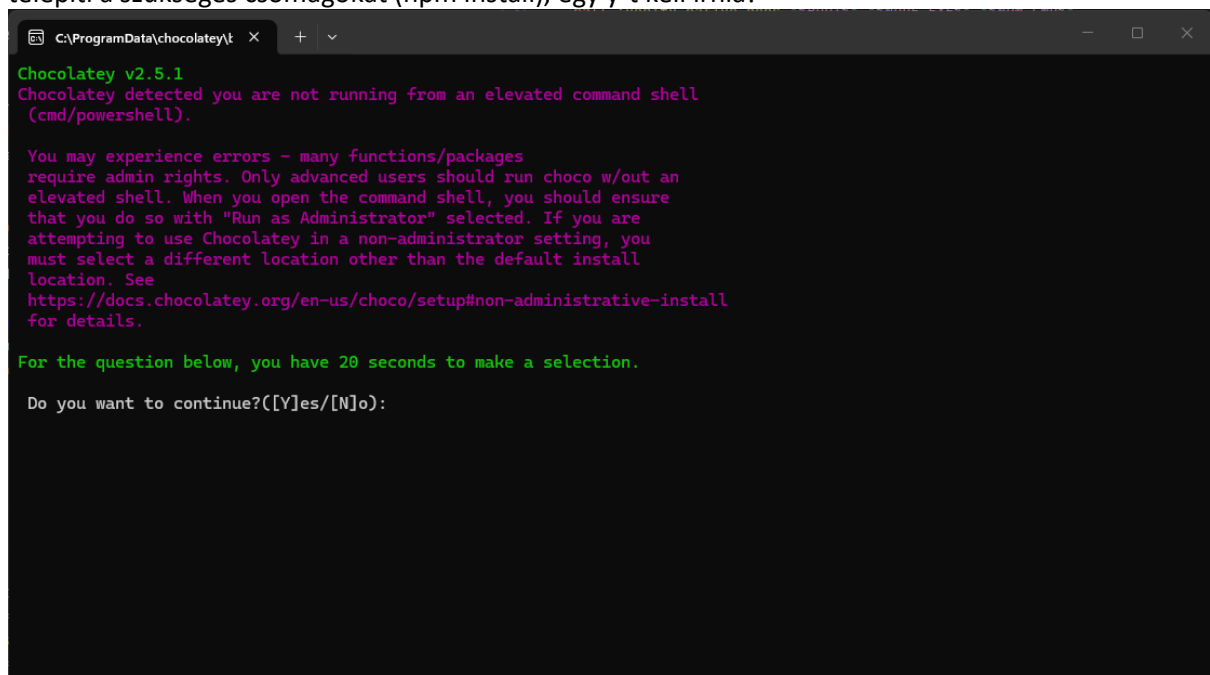
Run
cmd.exe /c start.bat
Checking Node.js...

```

szükség esetén telepíti a megfelelő Node verziót,



telepíti a szükséges csomagokat (npm install), egy y-t kell írnia!



```
C:\ProgramData\chocolatey\k  X + v
Chocolatey v2.5.1
Chocolatey detected you are not running from an elevated command shell
(cmd/powershell).

You may experience errors - many functions/packages
require admin rights. Only advanced users should run choco w/out an
elevated shell. When you open the command shell, you should ensure
that you do so with "Run as Administrator" selected. If you are
attempting to use Chocolatey in a non-administrator setting, you
must select a different location other than the default install
location. See
https://docs.chocolatey.org/en-us/choco/setup#non-administrative-install
for details.

For the question below, you have 20 seconds to make a selection.

Do you want to continue?([Y]es/[N]o): y
```

majd elindítja a backend és a frontend szolgáltatásokat.

Az első telepítés és csomag letöltés több percig is eltarthat, ez normális.

```
C:\ProgramData\chocolatey\k  X + v
Chocolatey v2.5.1
Chocolatey detected you are not running from an elevated command shell
(cmd/powershell).

You may experience errors - many functions/packages
require admin rights. Only advanced users should run choco w/out an
elevated shell. When you open the command shell, you should ensure
that you do so with "Run as Administrator" selected. If you are
attempting to use Chocolatey in a non-administrator setting, you
must select a different location other than the default install
location. See
https://docs.chocolatey.org/en-us/choco/setup#non-administrative-install
for details.

For the question below, you have 20 seconds to make a selection.

Do you want to continue?([Y]es/[N]o): y

Installing the following packages:
nodejs-lts
By installing, you accept licenses for the packages.
Downloading package from source 'https://community.chocolatey.org/api/v2/'
```

Sikeres indítás után a szerver(ek) futni fognak, a felület pedig a böngészőből elérhető a localhost:80 - as url-en.

```
ng serve --port 80 --host 0.0.0.0 X + v
polyfills.js | polyfills | 95 bytes |
Initial total | 220.26 kB

Lazy chunk files | Names | Raw size
chunk-01ZCKS7H.js | - | 947.47 kB
chunk-BTTAGBIS.js | home-component | 274.31 kB
chunk-7J5I7RDN.js | admin-component | 206.25 kB
chunk-0B2BJWDD.js | message-component | 149.44 kB
chunk-5NO63WS3.js | profile-component | 109.24 kB
chunk-L4ZYM672.js | - | 69.58 kB
chunk-74VEJ70Y.js | - | 51.24 kB
chunk-2W4YKVKX.js | leaderboard-page-component | 49.48 kB
chunk-VN5KXF53.js | product-page-component | 49.24 kB
chunk-23XBPU7X.js | settings-component | 45.67 kB
chunk-FE4UGP26.js | register-component | 44.04 kB
chunk-WVGS7UFG.js | - | 25.09 kB
chunk-3Z7IU74W.js | login-component | 20.11 kB
chunk-WEHNIATU.js | usersite-component | 19.04 kB
chunk-PQFONA4D.js | - | 15.24 kB
...and 12 more lazy chunks files. Use "--verbose" to show all the files.

Application bundle generation complete. [5.561 seconds] - 2026-05-07T20:22:18.395Z

Watch mode enabled. Watching for file changes...
NOTE: Raw file sizes do not reflect development server per-request transformations.
→ Local: http://localhost:80/
→ Network: http://192.168.1.89:80/
→ press h + enter to show help

Backend Server - call "C:\Use X + v
[dotenv@17.2.3] injecting env (18) from ..\.env -- tip: add observability to secrets: https://dotenvx.com/ops
🖼️ Serving images from: C:\Users\Levente\WebstormProjects\Team-Work\datas\images
Server running on http://localhost:3000
❌ Logger failed to write to disk: UNKNOWN: unknown error, open 'C:\Users\Levente\WebstormProjects\Team-Work\datas\Jsons\server-logs.json'
✅ ActiveUsers snapshot persisted { day: '2026-05-07', active_id: 1 }
❌ Logger failed to write to disk: UNKNOWN: unknown error, open 'C:\Users\Levente\WebstormProjects\Team-Work\datas\Jsons\server-logs.json'
✅ ActiveUsers synced to Supabase on shutdown { day: '2026-05-07' }
```

```
Admin Console - call "C:\Usei  X + v
[dotenv@17.2.3] injecting env (18) from ..\.env -- tip: ⚙ enable debug logging with { debug: true }
[dotenv@17.2.3] injecting env (18) from ..\.env -- tip: 🛡 prevent committing .env to code: https://dotenvx.com/precommi
t
Admin console ready. Type: help
ADMIN>
```

#### Hibakezelés

Ha az indítás közben hibaüzenet jelenik meg, ne zárja be az ablakot azonnal, hanem olvassa el a hiba okát.

Ha a telepítés valamiért nem tud automatikusan lefutni (pl. jogosultság vagy hálózati korlátozás), telepítse manuálisan a Node.js 20.20.x (x64) verziót, majd futtassa újra a start.bat-ot.

LINK: [ <https://nodejs.org/dist/v20.20.2/node-v20.20.2-x64.msi> ]

## 8.1 Rendszerkövetelmények

#### Minimum követelmények

Operációs rendszer: Windows 10 vagy Windows 11 (64-bit)

Processzor: 2 magos CPU (ajánlott 4 mag)

Memória (RAM): minimum 8 GB (ajánlott 16 GB)

Tárhely: minimum 2–4 GB szabad hely (függ a node\_modules méretétől)

Internetkapcsolat: szükséges az első telepítéshez és a csomagok letöltéséhez (ha nincs előre letöltve minden)

#### Szoftveres követelmények

Node.js + npm: a projekt a Node.js 20.20.x (x64) verzióval kompatibilis

A start.bat megpróbálja automatikusan telepíteni/ellenőrizni, de manuális telepítés is lehetséges.

Parancssor (CMD / PowerShell): szükséges a start.bat futtatásához

Böngésző: Chrome / Edge / Firefox (ajánlott Chrome vagy Edge)

## 10. Összegzés és továbbfejlesztési lehetőségek

Erre a projektre nagyon sok tervünk volt, kicsit részletesebben bele akartunk menni a különböző rendszerekbe eszközökbe, de sajnos nem jutott rá időnk. Az egész projektet nagyon megszerettük, és nagyon sok féle fejlesztési javaslatokat szedtünk össze. Így összegezve ez a program azoknak szól akik saját számítógépet, házimozit és saját eszközeiket akarja egybe látni, az értékét az eszközeinek, vagy az oldalon található AI segít összeállítani a megfelelő számítógépet és vagy a további oldalon található eszközöket.

### Továbbfejlesztési lehetőségek

AI rendszer fejlesztése: a Gemini helyett egy jóval erősebb, helyben futtatható (offline) AI bevezetése, ami több tudással, jobb pontossággal és alacsonyabb késleltetéssel segíti a felhasználót.

Részletes keresés bővítése a Home oldalon: a komponensekhez kapcsolódó benchmark adatok és teljesítményleszték integrálása (pl. processzor, videokártya), szűréssel és összehasonlítással.

Értesítések (Csengő) bővítése: több típusú értesítés (reportok, üzenetek, rendszerállapot-változások), jobb kategorizálás és fontos értesítések kiemelése.

Eszközök összekapcsolása (kábelek) több réteggel: részletesebb kapcsolati modell, ahol kiválasztható a kapcsolat típusa (HDMI, UTP, DVI, DisplayPort stb.), valamint a kompatibilitás ellenőrzése és vizuális ábrázolása.

Személyre szabhatóság növelése: profilkép állítása, egyedibb beállítások, akár saját színpaletta és téma (UI komponensek és háttér) testreszabása.